

Information Retrieval Notes

Course Notes

Academic Year 2026/2027

Contents

0	Introduction and Architecture of Information Retrieval	1
0.1	Definition and Motivations	1
0.2	The Two Macro-Objectives: Effectiveness and Efficiency . . .	3
0.3	Architecture of an Information Retrieval System	3
0.4	Language Properties and Retrieval Models	4
0.5	The Inverted Index	5
0.6	Index Compression Techniques	6
0.7	Learning-to-Rank (LTR) and the Non-Differentiability Challenge	7
0.8	The Shift Toward Neural Information Retrieval	7
1	Evaluation of Information Retrieval Systems	9
1.1	Search Engine Architecture	9
1.2	Evaluating Search Engine Quality	11
1.3	Measuring User Happiness	12
1.4	The Cranfield Paradigm and Offline Test Collections	14
1.5	Information Need vs. Query	18
1.6	Unranked Retrieval Measures	18
1.7	Rank-Based Measures (Binary Relevance)	21
1.8	Beyond Binary Relevance: Graded Relevance and NDCG . . .	26
2	Algorithmic Efficiency on Modern CPU Architectures	31
2.1	The Role of Efficiency in Information Retrieval	31
2.2	The Memory Hierarchy and Locality Principles	32
2.3	Forms of Parallelism and Pipeline Architecture	37
2.4	Superscalar Execution, Interleaving, and Structural Hazards .	38
2.5	Data Hazards and Serial Dependencies	41
2.6	Control Hazards, Speculative Execution, and Branch Prediction	42
2.7	Synthesis: From Hardware Constraints to Algorithmic Choices	45

3	Natural Language Processing and Text Representation	46
3.1	Graded Relevance Metrics (CG, DCG, NDCG)	46
3.2	Characteristics of Natural Language and Zipf’s Law	48
3.3	Data Acquisition and Web Parsing: urllib and BeautifulSoup .	50
3.4	Text Preprocessing Pipeline: Tokenization and Sentence Splitting	52
3.5	Vocabulary Construction and Counting	53
3.6	Morphological Normalization: Stemming vs. Lemmatization .	54
3.7	The Bag of Words (BoW) Model	56
3.8	N-grams: Word n-grams and Character n-grams	57
3.9	The Vector Space Model (VSM)	58
4	Retrieval Models and Ranking: Vector Space Model and BM25	60
4.1	Retrieval Models: Boolean vs Ranked Retrieval	60
4.2	Jaccard Similarity and its Limitations in Scoring	61
4.3	Term Weighting: Term-Document Matrices and TF-IDF . . .	62
4.4	Cosine Similarity and Length Normalization	63
4.5	SMART Notation and TF-IDF Variants	64
4.6	Best Match 25 (Okapi BM25)	65
4.7	Summary Conceptual Map	66
5	Search Engine Architecture: Indexing and Query Processing	67
5.1	System Architecture and the Two-Stage Retrieval Paradigm .	67
5.2	Scoring Models: The Dot Product Bridge to Dense Retrieval .	70
5.3	High-Performance Data Structures: Forward Index vs. Inverted Index	70
5.4	Inverted Index Anatomy, Memory Layout, and Compression .	72
5.5	Query Types and Query Expansion	73
5.6	Query Processing: TAAT vs. DAAT	74

Chapter 0

Introduction and Architecture of Information Retrieval

0.1 Definition and Motivations

Information Retrieval (IR) is the scientific and engineering discipline concerned with representing, storing, organizing, and accessing large collections of predominantly unstructured or semi-structured data to satisfy an **information need** expressed by a user [1, 2].

Definition 0.1.1 (Information Retrieval (Manning et al.)). Information Retrieval is finding material of an unstructured nature (usually text) that satisfies an information need from within large collections stored on computers.

IR is not limited to general-purpose Web search engines (Google, Bing): it underpins e-commerce and marketplace search, enterprise/desktop search, **RAG** (*Retrieval-Augmented Generation*) systems feeding Large Language Models with contextually relevant external documents, and recommender engines.

0.1.1 From Web Search to Agentic Search

The rise of conversational LLMs has not made Information Retrieval obsolete but has broadened its impact: modern LLMs continuously rely on core IR competencies (semantic query understanding, matching and ranking, contextual retrieval) to overcome their *knowledge cutoff*. In the **RAG** paradigm, the system intercepts the user’s request, queries an IR engine, injects the retrieved documents into the LLM’s context, and generates a response supported by verifiable citations.

Definition 0.1.2 (Agentic Search). **Agentic Search** is a dynamic retrieval process in which an intelligent agent autonomously decides which subsequent searches to conduct, based on partial evidence gathered: it inspects sources, identifies knowledge gaps (*information gaps*), formulates refined follow-up queries, and iterates until sufficient evidence has converged.

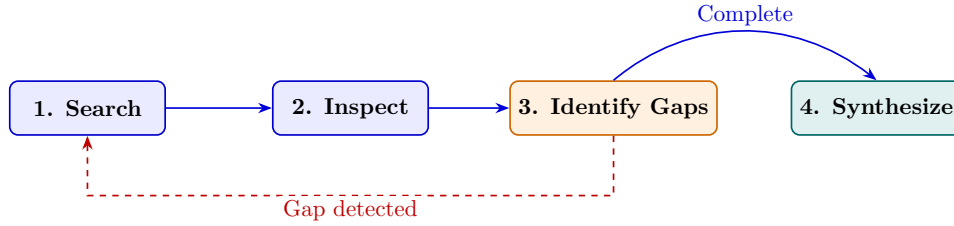


Figure 1: Adaptive feedback loop characterizing Agentic Search.

0.1.2 Information Retrieval vs. Data Retrieval

Dimension	Data Retrieval (SQL Databases)	Information Retrieval
Data structure	Rigidly schematized (tables, types, constraints)	Unstructured or semi-structured (free text, HTML)
Query language	Formal and deterministic (SQL)	Natural or keyword-based, often ambiguous
Matching	Exact Boolean	Partial and probabilistic (degrees of relevance)
Ordering	None unless explicit (ORDER BY)	Fundamental, driven by relevance estimation
Error tolerance	None	High (typos, synonymy, polysemy)

Table 1: Comparison between traditional Data Retrieval and Information Retrieval.

Relevance: The Cornerstone of IR

While in a database a record either satisfies or violates a Boolean predicate, in Information Retrieval the core challenge is estimating the degree of **relevance** with respect to the user’s information need: subjective, context-dependent, and evolving over time.

0.2 The Two Macro-Objectives: Effectiveness and Efficiency

A large-scale retrieval system pursues two objectives simultaneously, often in tension:

1. **Effectiveness:** how accurately the ranking satisfies the user’s information need.
2. **Efficiency:** computational cost, throughput, and response latency, typically on the order of 10–50 milliseconds per query over collections of billions of documents.

The efficiency of a modern search engine depends on its interaction with the underlying hardware architecture: pipelining and superscalar processors, memory hierarchy and access locality, SIMD vectorization, and parallel/distributed computing across clusters (*sharding*).

0.3 Architecture of an Information Retrieval System

An IR system enforces a strict decoupling between the **Offline Pipeline** (indexing and ranking model training) and the **Online Pipeline** (low-latency query serving), as depicted in Figure 2.

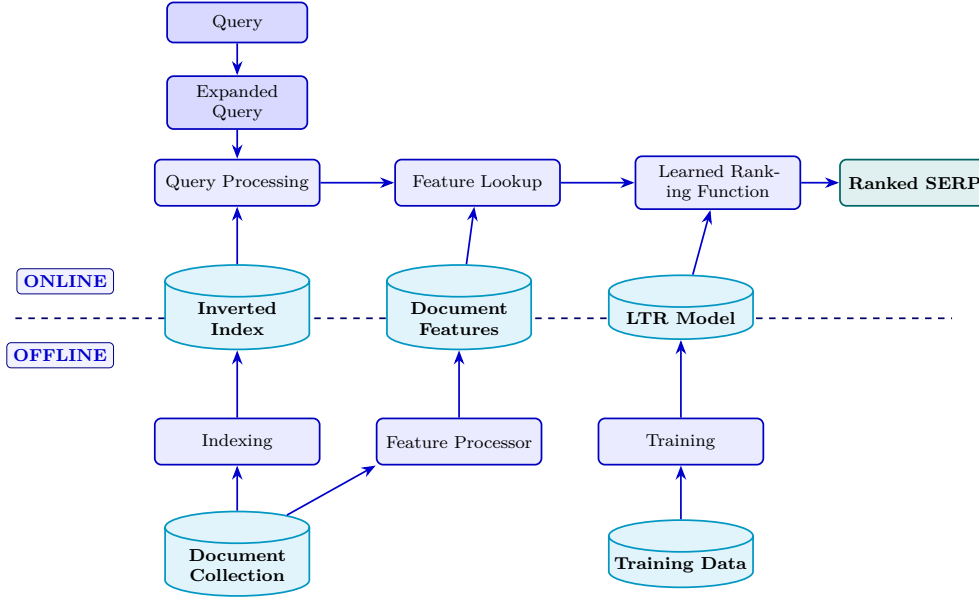


Figure 2: Grid architecture of a modern search engine (Online vs. Offline pipeline).

0.3.1 Text Processing Pipeline

Before indexing, every text stream goes through: **tokenization** (segmentation into lexical tokens), **normalization/case folding** (lowercasing, diacritics), **stop word filtering** (removal of high-frequency, low-discriminative words), and **stemming/lemmatization** (reducing an inflected form to its root or canonical lemma, e.g. "searching" → "search").

0.4 Language Properties and Retrieval Models

- **Zipf's Law:** the frequency $f(r)$ of a term is inversely proportional to its rank r in the frequency table, $f(r) \propto 1/r$: a few very frequent words account for most occurrences, while most terms occur only rarely (*long tail*).
- **Heaps' Law:** vocabulary size grows as a power law of the total number of words N in the collection, $|V| = k \cdot N^\beta$ (with $0.4 \leq \beta \leq 0.6$): the vocabulary never fully saturates.

The main retrieval models are the **Boolean Model** (exact match via logical operators, no relevance score), the **Vector Space Model** (queries and documents as TF-IDF weighted vectors, relevance via cosine similarity), and the **Probabilistic BM25 Model**, the de facto standard for lexical retrieval:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (1)$$

where $\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right)$ weighs term specificity, k_1 (typically 1.2–2.0) controls term-frequency saturation, and $b \in [0, 1]$ (typically ≈ 0.75) normalizes document length $|d|$ against the average length avgdl .

0.5 The Inverted Index

The **Inverted Index** reverses the logical orientation of document storage, realizing a mapping $t \mapsto \mathcal{P}_t$, where $\mathcal{P}_t$ is the sorted list of documents containing term t [1].

Definition 0.5.1 (Inverted Index). An Inverted Index consists of a **dictionary** (the sorted set of unique terms, with document frequency df_t and a pointer to the posting list) and, for each term, a **posting list**: an array sorted by increasing DocID, whose entries may include metadata such as local term frequency (tf) and positional offsets.

Example 0.5.1 (Reference Toy Collection). Consider a small collection of three documents about indexing and text search. Figure 3 shows the Inverted Index built for a few key terms: **a** appears in documents 1, 2, 3; **an** only in document 1; **index** and **inverted** in documents 1, 2; **query** only in document 3.

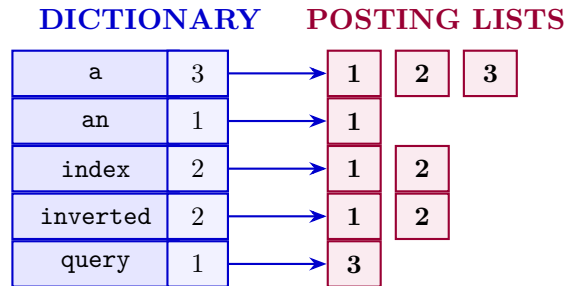


Figure 3: Structure of the Inverted Index: dictionary and posting lists.

Boolean constructs (AND, OR, NOT) and phrase search map directly to the advanced search interface of commercial search engines (Table 2).

IR Boolean Operator	Google Advanced Search Field	Syntax
Conjunction (AND)	<i>all these words</i>	<code>tri-colour rat terrier</code>
Phrase search	<i>this exact word or phrase</i>	<code>"rat terrier"</code>
Disjunction (OR)	<i>any of these words</i>	<code>miniature OR standard</code>
Negation (NOT)	<i>none of these words</i>	<code>-rodent -"Jack Russell"</code>
Numeric range	<i>numbers ranging from...to</i>	<code>10..35 kg</code>

Table 2: Mapping between IR Boolean operators and Google Advanced Search.

0.6 Index Compression Techniques

Inverted Index compression dramatically reduces memory footprint (both in RAM and on disk), converting an I/O bottleneck into a CPU decompression workload, which is generally much faster.

0.6.1 Binary Interpolative Coding

Binary Interpolative Coding (Moffat and Stuiver [3]) compresses strictly increasing sequences of integers (the DocIDs of a posting list) by recursively partitioning them: given a sublist of n integers known to lie within $[l, h]$, the middle element x_m (with $m = \lfloor (n+1)/2 \rfloor$) is encoded using $\lceil \log_2 R \rceil$ bits, where $R = h - l - n + 1$ is the number of admissible values, and the recursion proceeds separately on the two halves. When DocIDs are densely clustered, R tends toward 1 and many elements are encoded at near-zero cost.

0.6.2 Dictionary-Based Compression: LZ77

For general text, the **Lempel-Ziv '77** (LZ77) algorithm compresses via a sliding window, replacing lexical repetitions with triples composed of (offset, length, next_char) referring to earlier occurrences in the window. This family of algorithms underlies widely adopted tools such as `gzip`, `7-Zip`, and `LZ4`.

0.7 Learning-to-Rank (LTR) and the Non-Differentiability Challenge

Candidates retrieved by the Inverted Index are passed to a machine learning re-ranking stage known as **Learning-to-Rank** (LTR) [4]. The training dataset associates each query with a list of documents carrying graded relevance judgments $y \in \{0, \dots, 4\}$ (from irrelevant to perfectly relevant); a model computes a score $s_i = f_\theta(\vec{x}_{q,d_i})$ for each query-document pair, and the final list is obtained by sorting in decreasing order of score.

The Cornerstone of LTR

The goal is to learn the ranking, not the label: the objective is not to accurately estimate the single value y , but the optimal relative ordering, maximizing cumulative metrics such as **NDCG** or **MAP**.

Because the sorting operation is discrete and step-wise, the gradient of ranking metrics with respect to model parameters is zero almost everywhere: gradient descent cannot be applied directly. The literature has developed three families of solutions:

1. **Pointwise:** treats each document as an independent instance (regression or classification on y), ignoring the inherently comparative nature of ranking.
2. **Pairwise** (e.g. RankNet): compares pairs of documents, modeling the probability that d_i precedes d_j with a differentiable sigmoid.
3. **Listwise** (e.g. LambdaMART): optimizes the entire list via surrogate gradients λ_{ij} , weighted by the actual NDCG change resulting from swapping d_i and d_j ; combined with gradient-boosted decision trees, it was for over a decade the standard in commercial search engines.

0.8 The Shift Toward Neural Information Retrieval

Since 2018, Transformer-based models (BERT) have ushered in the era of Neural Information Retrieval [5, 6], pre-trained via **Masked Language Modeling**. Neural retrieval approaches fall into two families:

- **Interaction-based (Cross-Encoders):** query and document are concatenated and jointly processed by a Transformer, capturing deep

interactions at high computational cost ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$); used only as second-stage re-rankers over a small candidate set.

- **Representation-based (Bi-Encoders):** query and document are independently projected into a vector space by two distinct encoders; document vectors can be pre-computed offline, reducing online matching to a dot product or an approximate nearest-neighbor search.

Among representation-based models, three families of embeddings are distinguished: **dense single-vector** (a single dense vector per document/query, queried with k-ANN graph indexes such as HNSW), **dense multi-vector** (e.g. ColBERT [5], one vector per token with *Late Interaction*/MaxSim aggregation, $S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T)$), and **sparse single-vector** (e.g. SPLADE [6], interpretable by design and more space/time efficient than multi-vector).

k-Approximate Nearest Neighbor Search

For collections of millions of documents represented as high-dimensional dense vectors, exact retrieval of the k most similar to the query is computationally prohibitive: index structures for **approximate nearest-neighbor search** (k -ANN) are used instead, trading a minimal loss of accuracy for orders-of-magnitude speedups. **TusKANNy** (<https://tuskanny.com/>), developed by ISTI-CNR and the University of Pisa, is a high-performance open-source example written in Rust.

Chapter 1

Evaluation of Information Retrieval Systems

1.1 Search Engine Architecture

Before diving into evaluation methodology, it is instructive to understand the high-level block diagram of a modern search engine, since each architectural component introduces measurable quality dimensions.

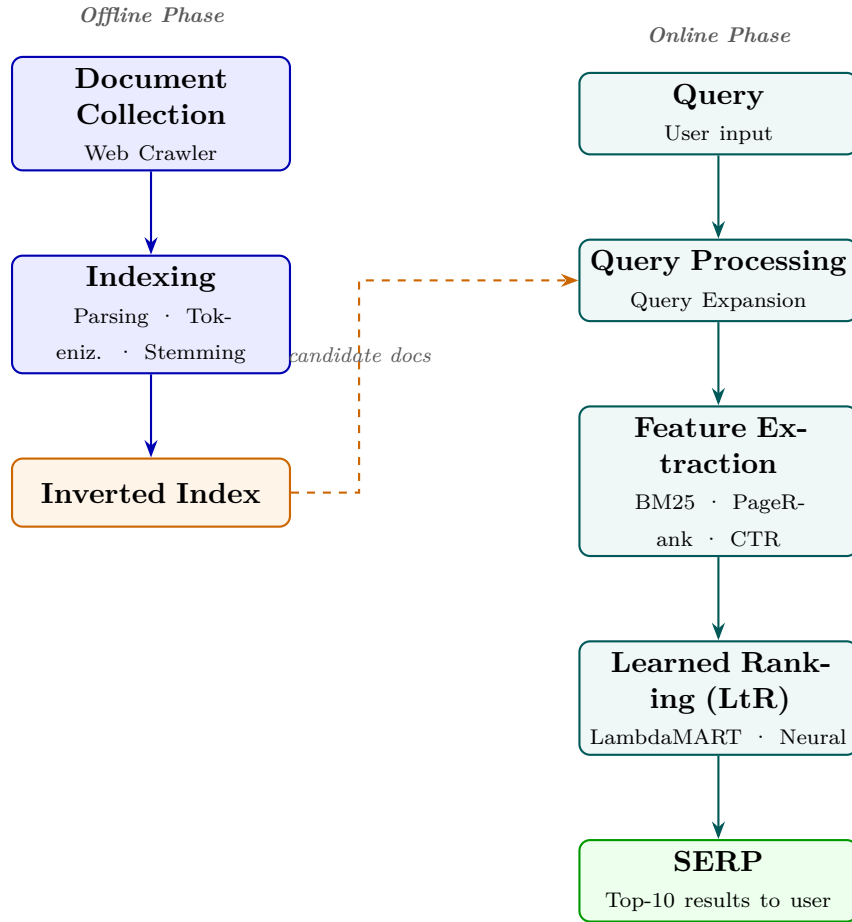


Figure 1.1: High-level block diagram of a modern search engine: offline indexing pipeline (left) and online query processing and ranking pipeline (right).

The main components are:

- **Web Crawler / Document Collection:** An automated agent that continuously discovers and downloads documents from the Web (or a closed corpus), feeding the indexing pipeline.
- **Parsing & Tokenization:** Documents are linguistically analyzed: HTML is stripped, text is tokenized into terms, stop-words may be removed, and stems or lemmas are computed.
- **Inverted Index:** The core data structure mapping each term to the list of documents containing it (posting list), together with positional and frequency information.

- **Query Processing & Query Expansion:** An incoming user query is tokenized with the same pipeline, possibly enriched with synonyms, related terms, or session context to improve recall.
- **Feature Extraction:** For each candidate document, a rich feature vector is assembled (BM25 score, PageRank, freshness, anchor text signals, click history, ...).
- **Learned Ranking Function (Learning to Rank):** A machine-learned model (e.g., LambdaMART, neural ranker) combines the features to produce a final relevance score used to sort the SERP.
- **SERP (Search Engine Results Page):** The final ordered list of results displayed to the user, possibly augmented with featured snippets, knowledge panels, and ads.

1.2 Evaluating Search Engine Quality

How do we evaluate such a complex, multi-component system? The answer lies in separating two complementary dimensions—**efficiency** and **effectiveness**—each requiring distinct measurement methodologies [1, 2].

- **Efficiency** concerns the *speed and resource cost* of processing: how fast can we index documents, and how quickly can we answer queries at scale?
- **Effectiveness** concerns the *quality of the answers*: are the returned documents genuinely useful to the user?

Rigorous performance evaluation is essential for designing, fine-tuning, and deploying Information Retrieval systems [1, 2]. Search engine quality is thus evaluated across these two complementary dimensions:

1. Efficiency Metrics:

- *Indexing throughput:* Number of documents processed and incorporated per hour.
- *Incremental indexing:* Ingesting new document streams (e.g., +10 000 products daily) without downtime.
- *Search latency and throughput:* Query processing response time (demanding 95th/99th percentile latencies within 10–50 ms) and CPU/memory utilization under workloads of thousands of queries per second over multi-million or multi-billion document collections.

2. Effectiveness Metrics and Auxiliary Services:

- Quality of related product recommendations, query spell-checking accuracy, and snippet generation readability.
- **Intrinsic ranking effectiveness:** Measures the engine’s core capability to satisfy users by ranking relevant documents at the top of the Search Engine Results Page (SERP).

1.3 Measuring User Happiness

In commercial Web search engines, the overarching objective is maximizing **User Happiness**. Because happiness is an internal cognitive state that cannot be directly queried, production environments rely on observable online behavioral proxies:

- **Clicks and Click-Through Rate (CTR):** The volume and proportion of clicks received by returned results.
 - *Caveat:* Misleading titles and clickbait summaries artificially inflate clicks without truly satisfying the user.
 - *Zero-Click Searches:* When is “no clicks” good news? For factual queries (e.g., weather forecasts, flight schedules, currency conversions, or dictionary definitions), answers rendered directly via *Featured Snippets* or *Knowledge Panels* satisfy the user immediately without requiring any downstream click.
- **Purchases and Economic Conversions:** In e-commerce, the percentage of search sessions resulting in transactions and total monetary spend serve as key indicators.
- **Retention and Repeat Visits:** The fraction of users returning to the search engine within a week or month.
- **Dwell Time:** The duration an attendee spends exploring a clicked page before navigating back to the SERP (*bouncing back*). Extended dwell time indicates genuine engagement, whereas near-instant returns signal dissatisfaction.

Evaluation in the Era of Generative AI and LLMs

The rise of generative conversational search engines and Large Language Models (LLMs) fundamentally alters traditional metrics such as CTR and Dwell Time: users consume direct synthesized summaries rather than navigating outgoing links. Consequently, factual grounding, hallucination minimization, and source attribution fidelity supplant traditional link click telemetry.

Several critical methodological debates have emerged regarding the evaluation of these systems:

- **Number of conversational turns as a failure proxy:** In a generative dialogue system, an excessive number of turns required to satisfy an information need often signals that the system failed to comprehend the user’s initial intent, forcing repeated clarifications or reformulation loops. Traditional Dwell Time thus yields to conversational length and session efficiency metrics.
- **Prompting variance and reproducibility:** Unlike keyword queries in classical IR, slight changes in prompt wording to a generative LLM can induce substantial variance in response quality. This intrinsic sensitivity severely complicates standardized, reproducible benchmarking.
- **Ontological difference between retrieval and synthesis:** Traditional IR systems *retrieve* and rank preexisting documents from a known collection; generative LLMs *synthesize* original probabilistic text. In retrieval, relevance is evaluated against a fixed corpus and gold-standard ground truth; in synthesis, verifying factual validity and correctness requires entirely new semantic evaluation frameworks.
- **Retrieval-Augmented Generation (RAG):** The RAG paradigm addresses hallucination risks by conditioning generative synthesis on documents fetched via classical IR. Consequently, evaluation requires decoupled yet unified assessment: measuring both retrieval quality and synthesis fidelity.

1.4 The Cranfield Paradigm and Offline Test Collections

While online signals offer valuable telemetry, they are noisy, subject to position bias, and unsuitable for comparing algorithmic improvements offline prior to production deployment.

To situate the Cranfield approach historically: Cyril Cleverdon conducted his seminal experiments in the **1960s**, working with *physical* library systems—card catalogues and aeronautical engineering records at the Cranfield Institute of Technology—long before digital text retrieval was computationally feasible. His insight was to construct a controlled experimental collection with exhaustive relevance judgments, providing the first rigorous laboratory for IR comparison. This setting, humble by today’s standards, established a scientific template still followed six decades later.

To establish reproducible, controlled evaluation, IR research adheres to the **Cranfield Paradigm**, pioneered by Cyril Cleverdon in the 1960s [7].

Definition 1.4.1 (Test Collection). A standard Information Retrieval test collection is a formal triplet $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ consisting of:

1. A benchmark document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. A benchmark suite of queries $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. A set of relevance judgments (*qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, assigning a ground-truth relevance assessment (binary or graded) to each query-document pair.

1.4.1 Combinatorial Scale Bottlenecks

Exhaustively assessing all query-document pairs is physically infeasible for realistic corpus sizes. Consider a representative benchmark with:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documents, } |\mathcal{Q}| = 50\,000 \text{ queries}$$

The full Cartesian product comprises:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 50\,000 = 2.5 \times 10^{11} \text{ pairs } \quad (0.25 \text{ trillion judgments})$$

Assuming an experienced human assessor evaluates one document in 2.5 seconds:

$$\begin{aligned} \text{Total Time} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ seconds} \\ &\approx 173\,611\,111 \text{ person-hours} \end{aligned}$$

This would require tens of thousands of human working years! Consequently, benchmark datasets must restrict assessment to a sampled subset of candidate documents.

1.4.2 Origins of Test Queries

A natural question is: *where do the queries in a benchmark test suite come from?* The answer differs significantly depending on the evaluation context:

- **Web search engines — Query Logs:** Large-scale industrial benchmarks (such as those used in Microsoft TREC tracks or the AOL query log dataset) sample queries directly from *real user query logs*. This ensures that the test suite faithfully reflects the actual distribution of user information needs, including rare and long-tail queries.
- **Classic laboratory scenarios — Expert-drafted lists:** In pre-Web IR benchmarks (Cranfield, TREC ad hoc tracks), domain experts *manually composed* a set of information need statements (called *topics*). Each topic is a verbose narrative describing the underlying information need, which is then abbreviated into a short keyword query for system input.
- **What is *not* done — Random queries from documents:** One might naively suggest generating queries by sampling random terms from documents in the collection. This approach is **explicitly avoided** in IR evaluation: randomly sampled terms do not resemble real user intents, would introduce severe artificial biases, and would not generalize to the retrieval scenarios that actually matter to end users.

1.4.3 Crowdsourcing: Opportunities and Pitfalls

One cost-effective alternative delegates judging to online micro-task platforms (e.g., Amazon Mechanical Turk). However, empirical literature indicates that crowdsourced annotations suffer from:

- Substantially higher variance compared to domain experts;
- Inconsistent quality and exposure to adversarial spamming;
- A requirement for robust filtering mechanisms (majority voting, worker reputation scoring, Cohen’s κ inter-rater reliability thresholds).

A more recent and increasingly adopted alternative is **LLM-as-a-Judge**:

- **LLM-as-a-Judge:** State-of-the-art language models (such as GPT-4-class models) are prompted with structured evaluation rubrics to assign relevance grades to query-document pairs. This approach offers substantial *economic scalability*—millions of judgments can be obtained at a fraction of the cost of human crowdworkers. However, it introduces well-documented *intrinsic biases*: **length bias** (longer, more verbose responses tend to receive higher scores regardless of content quality), **self-preference bias** (a model tends to rate its own outputs more favorably), and **position bias** (the order in which candidates are presented influences the judgment). Active research is ongoing to quantify and mitigate these distortions.

1.4.4 The TREC Infrastructure

Organized by the National Institute of Standards and Technology (NIST), the **Text REtrieval Conference (TREC)** provides the foundational infrastructure for large-scale IR experimentation. TREC is organized into specialized thematic *tracks* reflecting distinct information access tasks (Web search, biomedical IR, conversational search, deep learning retrieval). The output produced by a participant system executing a retrieval task on a test collection is termed a **Run**.

Table 1.1 documents the dimensional evolution of classical benchmark collections across Information Retrieval history [1], emphasizing the massive scale increase that necessitated the creation of TREC.

Collection	Documents (N)	Queries	Size (MB)	Terms/Doc	Assessments (Q-D)
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	> 10 000
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Exhaustive
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Typical)	740 000	200	2 000 (2 GB)	89–3 543	> 100 000

Table 1.1: Comparative overview of classical historical evaluation test corpora.

As evident from the empirical data, early corpora from the 1960s and 1970s (such as Cranfield and Medline) comprised only a few thousand records and 1–2 MB, allowing exhaustive manual judging. With the advent of realistic Web-scale corpora containing hundreds of thousands of documents

and gigabytes of text like TREC, the combinatorial evaluation space exploded, mandating selective sampling strategies.

1.4.5 The Pooling Technique

To select which documents human assessors should judge without evaluating the entire corpus, TREC standardizes the **Pooling** methodology (Figure 1.2):

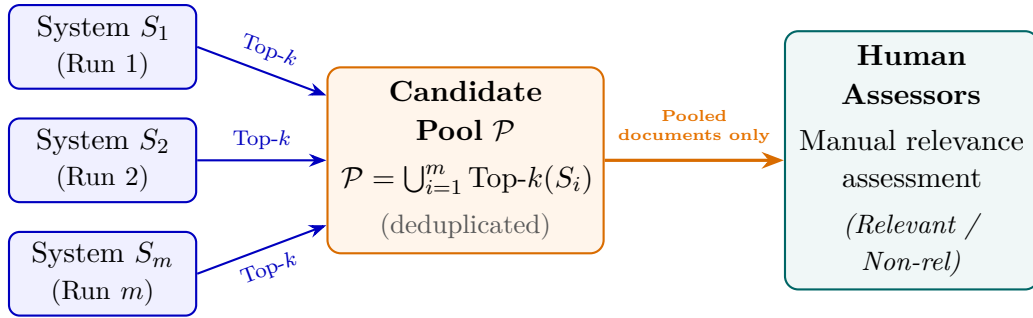


Figure 1.2: The Pooling technique for selecting candidate documents for human judgment.

1. Collect ranked result lists from m diverse retrieval systems for a given topic.
2. Extract the top- k ranked documents from each run (typically $k = 50$ or $k = 100$).
3. Take the **union** of these candidate sets, deduplicating documents to build pool $\mathcal{P}$.
4. Human judges assess relevance strictly for documents inside $\mathcal{P}$.

The Unjudged Document Assumption

All documents in the corpus not present in the pool (*unpooled*) are formally assumed to be **non-relevant** ($r = 0$). While this assumption may theoretically penalize innovative systems surfacing relevant documents overlooked by historical pool contributors, NIST experiments confirm that sufficiently diverse pools maintain stable, fair, and reliable system rankings.

1.5 Information Need vs. Query

A crucial principle in evaluation separates the underlying need from its textual expression:

- **Information Need:** The user's actual cognitive desire. Example: "*My swimming pool bottom is becoming black and needs to be cleaned*".
- **Query:** The keyword shorthand submitted to the system. Example: "pool cleaner".

Property 1.5.1 (Evaluation Principle). *Document relevance must always be assessed with respect to the user's underlying **Information Need**, never purely against the superficial keywords of the query string.*

1.6 Unranked Retrieval Measures

In unranked evaluation contexts (e.g., Boolean retrieval), the engine returns an unordered set of documents. Performance is derived from the standard contingency table (Table 1.2):

	Document Relevant	Document Non-Relevant
Document Retrieved	True Positive (<i>TP</i>)	False Positive (<i>FP</i>)
Document Not Retrieved	False Negative (<i>FN</i>)	True Negative (<i>TN</i>)

Table 1.2: Contingency matrix for binary query-document classification.

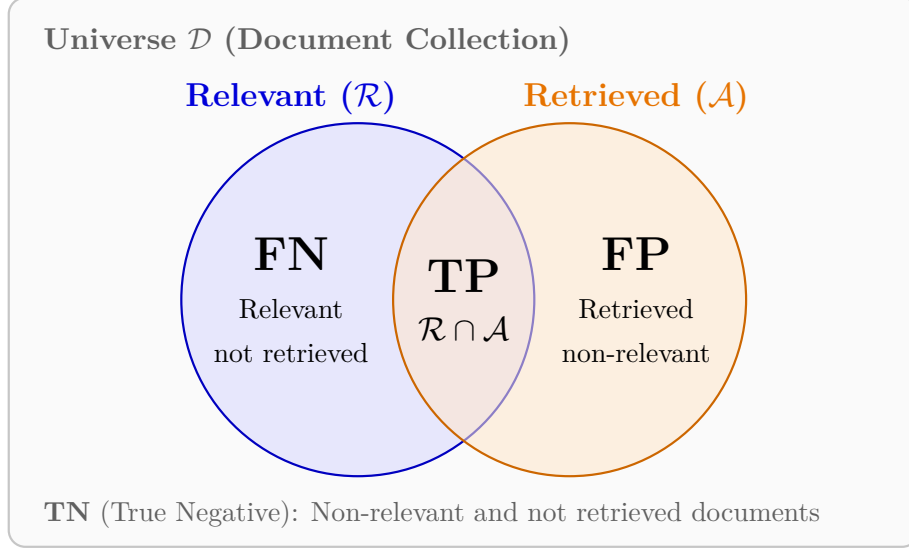


Figure 1.3: Set-theoretic Venn diagram of document retrieval and confusion regions.

1.6.1 Precision, Recall, and the Precision-Recall Trade-off

Definition 1.6.1 (Precision (P)). The fraction of retrieved documents that are relevant:

$$P = \frac{TP}{TP + FP} = P(\text{relevant} \mid \text{retrieved}) \quad (1.1)$$

Definition 1.6.2 (Recall (R)). The fraction of relevant documents in the collection that were successfully retrieved:

$$R = \frac{TP}{TP + FN} = P(\text{retrieved} \mid \text{relevant}) \quad (1.2)$$

An inherent **trade-off** governs Precision and Recall:

- Returning the entire document repository (Retrieved = $\mathcal{D}$) guarantees $R = 100\%$, but drops precision near zero ($P \approx 0$).
- Conversely, returning only the single most confident document can yield $P = 100\%$, at the expense of dismal recall.

1.6.2 The Harmonic Mean: F-Measure (F_1 -Score)

To balance Precision and Recall into a single scalar metric that penalizes extreme disparity, the **F-Measure** calculates their **harmonic mean**:

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (1.3)$$

Property 1.6.1 (Harmonic Mean Properties). *The harmonic mean exhibits key mathematical properties for evaluation:*

1. *It is always upper-bounded by both the geometric and arithmetic means:*

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. *When the two operands diverge significantly, the harmonic mean is heavily drawn toward the **minimum** rather than the arithmetic average.*

Example 1.6.1 (Edge Behavior of F-Measure). Consider a degenerate retrieval system returning all documents: $R = 1.0$ and $P = 0.0001$.

- The arithmetic mean yields an artificially inflated score:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- The harmonic mean (F_1) penalizes the system:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

This illustrates why the harmonic mean is mathematically favored over the arithmetic mean.

1.6.3 The Weighted F_β -Measure

When asymmetric priority is given to Precision versus Recall, a weighting parameter $\alpha \in [0, 1]$ is introduced:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (1.4)$$

Setting $\alpha = \frac{1}{1+\beta^2}$ leads to the standard F_β formulation:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (1.5)$$

- Values of $\beta < 1$ (e.g., $\beta = 0.5$) emphasize **Precision** over Recall (favored in Web search).
- Values of $\beta > 1$ (e.g., $\beta = 2.0$) emphasize **Recall** over Precision (crucial in legal discovery, patent search, and clinical meta-analyses).

1.7 Rank-Based Measures (Binary Relevance)

Modern search engines return ranked lists ordered by descending relevance scores. Furthermore, Web users rarely look beyond the first page of results ($k = 10$ or $k = 30$).

1.7.1 Precision@k ($P@k$) and Recall@k ($R@k$)

Definition 1.7.1 (Precision@k). Given a fixed rank cutoff $k \in \mathbb{N}^+$, Precision@k is the fraction of relevant documents within the top k positions:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (1.6)$$

where $\text{rel}_i \in \{0, 1\}$ represents the binary relevance of the document at rank i .

Example 1.7.1 (Computing Precision@k). Consider a ranked list with binary relevance sequence for the top 5 positions $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (note that $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$

The Distortion of $P@k$ When Averaged Across Queries

While $P@k$ is intuitive for estimating top-page result quality, it suffers from a critical methodological flaw when averaged across heterogeneous queries:

- If a specific query q_A possesses only $R_A = 2$ relevant documents in the entire corpus, the theoretical maximum possible $P@10(q_A)$ is capped at $\frac{2}{10} = 0.20$, even if a flawless engine places both relevant documents at ranks 1 and 2!
- Averaging raw $P@10$ across queries penalizes the retrieval engine for the intrinsic scarcity of relevant documents for specific information needs, rather than algorithmic ranking errors.

Similarly, the complementary measure $Recall@k$ ($R@k$) evaluates the fraction of all relevant documents recovered within rank k :

$$Recall@k = \frac{1}{R} \sum_{i=1}^k rel_i \quad (1.7)$$

1.7.2 Average Precision (AP)

Average Precision (AP) summarizes the entire Precision-Recall curve into a single scalar score for a given query, evaluating instantaneous precision at each rank position where recall increases.

Definition 1.7.2 (Average Precision). For a ranking of N documents corresponding to a query with R total relevant documents in the collection:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot rel_k \quad (1.8)$$

Property 1.7.1 (Geometric Interpretation and the Role of $1/R$). *Average Precision carries a direct analytical meaning:*

1. *Each time a relevant document is encountered at rank k , recall increases by a discrete increment $\Delta Recall = \frac{1}{R}$. AP accumulates the rectangular areas $P@k \cdot \Delta Recall$, representing the **discrete numerical approximation of the Area Under the Precision-Recall Curve** (AUC-PR).*
2. *Normalizing by R (the total number of relevant documents in the corpus) ensures that relevant documents omitted from the top- N candidate pool*

implicitly contribute zero precision, heavily penalizing systems with incomplete recall.

	Search Engine Output Order				
Rank Position (k):	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
Document:	Rel	NonRel	Rel	NonRel	Rel
Precision@ k :	1.00	—	0.67	—	0.60

Figure 1.4: Schematic representation of Average Precision calculation over 5 documents with $R = 3$ total relevant items.

Example 1.7.2 (Basic Calculation with $R = 3$ Relevant Documents). Consider the ranking scenario in Figure 1.4 with $R = 3$ total relevant documents appearing at ranks $k_1 = 1$, $k_2 = 3$, and $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

The resulting Average Precision is:

$$AP = \frac{1}{3} \left(1.00 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (2.267) \approx 0.756 \quad (75.6\%) \quad (1.9)$$

Example 1.7.3 (Comparative Analysis Between Two Rankings). Consider a query associated with $R = 6$ relevant documents in the collection. Two competing search algorithms output the following top-10 ranked results:

- **Ranking #1:** Relevant items concentrated early in the list:

$\langle \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{NonRel}, \text{Rel} \rangle$

- **Ranking #2:** Relevant items delayed and scattered:

$\langle \text{NonRel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel} \rangle$

Table 1.3 presents the step-by-step evolution of Precision and Recall across all 10 ranks.

Rank (k)	Ranking #1			Ranking #2		
	Item	Recall	Precision	Item	Recall	Precision
1	Rel	0.17	1.00	NonRel	0.00	—
2	NonRel	0.17	—	Rel	0.17	0.50
3	Rel	0.33	0.67	NonRel	0.17	—
4	Rel	0.50	0.75	NonRel	0.17	—
5	Rel	0.67	0.80	Rel	0.33	0.40
6	Rel	0.83	0.83	Rel	0.50	0.50
7	NonRel	0.83	—	Rel	0.67	0.57
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	Rel	0.83	0.56
10	Rel	1.00	0.60	Rel	1.00	0.60
Average Precision (AP)			0.78	0.52		

Table 1.3: Detailed comparative evaluation of Average Precision between Ranking #1 and Ranking #2.

The resulting AP scores evaluate to:

$$\begin{aligned}
 AP(\text{Ranking \#1}) &= \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78} \\
 AP(\text{Ranking \#2}) &= \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}
 \end{aligned}$$

Although both systems achieve identical overall recall ($R = 100\%$ across 10 retrieved documents), Ranking #1 obtains a substantially higher score (0.78 vs 0.52) because it rewards front-loading relevant items.

1.7.3 Mean Average Precision (MAP)

To quantify overall effectiveness across a suite of $|\mathcal{Q}|$ benchmark queries, we compute the arithmetic mean of their respective AP scores:

Definition 1.7.3 (Mean Average Precision).

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \quad (1.10)$$

Property 1.7.2 (Macro-Averaging vs. Micro-Averaging). *MAP formally implements a **Macro-Averaging** scheme: each query carries identical statistical weight $1/|\mathcal{Q}|$.*

- *In contrast, Micro-Averaging (pooling all instantaneous precisions across all queries and dividing by the total sum of relevant documents) would be overwhelmingly dominated by a handful of broad queries (e.g., "movies" or "news", which often match thousands of relevant documents), drowning out performance on specific long-tail queries.*
- *Macro-Averaging ensures that every individual information need receives equal representation in the final benchmark metric.*

Example 1.7.4 (Computing MAP Over Two Queries). Consider an evaluation suite consisting of two distinct queries q_1 and q_2 :

- **Query 1 ($R_1 = 5$ total relevant documents):** The engine returns 5 relevant items at ranks $k \in \{1, 3, 6, 9, 10\}$ with precisions 1.00, 0.67, 0.50, 0.44, and 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 ($R_2 = 3$ total relevant documents):** The engine returns 3 relevant items at ranks $k \in \{2, 5, 7\}$ with precisions 0.50, 0.40, and 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

The overall Mean Average Precision across both queries is:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \quad (1.11)$$

1.7.4 Truncated Metrics: AP@K and MAP@K

In Web search and Recommender Systems, evaluating AP across an entire collection is impractical (the true total count R of relevant items is unknown) and disconnected from user browsing habits. Engines thus adopt rank-truncated formulations (**MAP@K**):

Definition 1.7.4 (AP@K and MAP@K). Given a fixed rank cutoff K (typically $K = 10$ or $K = 20$), Average Precision at K for query q_i is defined as:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot \text{rel}(q_i, k) \quad (1.12)$$

where the normalizing denominator K_i reflects the number of relevant items within the top K :

$$K_i = \sum_{k=1}^K \text{rel}(q_i, k) \quad (\text{or } K_i = \min(R_i, K)) \quad (1.13)$$

The global **MAP@K** across query suite $\mathcal{Q}$ evaluates to:

$$\text{MAP@K}(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} \text{AP@K}(q_i) \quad (1.14)$$

This formulation resolves the fundamental flaw of raw $P@K$: by normalizing by K_i instead of a rigid K , a query with only 2 relevant documents that places both in the top 2 slots attains $\text{AP@10} = 1.0$ rather than being artificially capped at 0.20.

1.7.5 Mean Reciprocal Rank (MRR)

In tasks such as known-item search, navigational queries (e.g., locating an institutional homepage), or factoid retrieval, the user is interested solely in finding the **first correct document**. User effort is inversely proportional to the rank of that first hit:

Definition 1.7.5 (Reciprocal Rank and MRR). Let $\text{rank}_1(q)$ denote the rank position of the first relevant document returned for query q . The **Reciprocal Rank** (RR) is:

$$\text{RR}(q) = \frac{1}{\text{rank}_1(q)} \quad (1.15)$$

The **Mean Reciprocal Rank** (MRR) across all queries is:

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (1.16)$$

If the first relevant hit appears at rank 1, the score is 1.0; if at rank 2, it drops to 0.5; if delayed to rank 10, it falls to 0.1.

1.8 Beyond Binary Relevance: Graded Relevance and NDCG

In practical Web and commercial applications, binary judgments prove inadequate. Consider the following real-world Google search query:

Query: "information retrieval unipi"

A commercial SERP returns pages exhibiting distinct qualitative levels of utility:

1. **Course DidaWiki (Instructor, CFU, syllabus, lecture notes):** The primary resource fully satisfying user search intent $\implies$ **Highly Relevant** ($r = 3$).
2. **Official UniPi Valutami Portal (Exam registration and administrative syllabus):** Authoritative institutional record but lacks study material $\implies$ **Relevant** ($r = 2$).
3. **Master's Degree PDF Brochure:** Contains only a brief mention of the course $\implies$ **Marginally Relevant** ($r = 1$).
4. **Unrelated articles or campus news:** $\implies$ **Irrelevant** ($r = 0$).

While all top three pages are pertinent, user satisfaction depends heavily on displaying the most informative page at rank 1. To capture this nuance, IR formalizes multi-grade relevance scales ($r \in \{0, 1, 2, 3\}$ or $\{0, \dots, 4\}$).

The **Discounted Cumulative Gain** (DCG) framework, introduced by Järvelin and Kekäläinen [8], rests on two assumptions:

1. Highly relevant documents are much more useful than marginally relevant ones.
2. The lower a document appears in the ranking, the less likely the user is to examine it; hence, gain must be discounted with descending rank.

1.8.1 Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)

Definition 1.8.1 (Cumulative Gain at Rank p). Given a vector of graded relevance values $\mathbf{r} = [r_1, r_2, \dots, r_p]$, the Cumulative Gain is the unweighted sum:

$$CG_p = \sum_{i=1}^p r_i \quad (1.17)$$

CG_p is position-insensitive: swapping a highly relevant document at rank 1 with one at rank 10 leaves CG_p unchanged.

Definition 1.8.2 (Discounted Cumulative Gain (Standard Formulation)). Standard DCG applies a logarithmic discount based on rank i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (1.18)$$

Behavioral Meaning of the Logarithmic Discount

The discount factor $\frac{1}{\log_2(i)}$ reflects the exponential decay of user visual attention under cascade reading models:

- At rank $i = 4$, the discount factor is $\frac{1}{\log_2(4)} = \frac{1}{2}$: perceived informative utility is cut in half relative to rank 1.
- At rank $i = 8$, the factor drops to $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- At rank $i = 16$, the factor falls to $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Commercial search engines (Google, Bing) frequently utilize the **exponential formulation**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (1.19)$$

Summation Index Clarification

Note the index shift in the logarithm denominator:

- In Equation (1.18), r_1 is separated to prevent division by $\log_2(1) = 0$, while the summation starts at $i = 2$ with $\log_2(i)$.
- In Equation (1.19), the summation starts directly at $i = 1$ by taking $\log_2(i + 1)$ ($\log_2(1 + 1) = \log_2(2) = 1$).
- Furthermore, numerator $2^{r_i} - 1$ non-linearly rewards top-tier relevance ($r = 3 \implies 2^3 - 1 = 7$, vs. $r = 1 \implies 2^1 - 1 = 1$), severely penalizing any demotion of stellar results.

1.8.2 Normalized Discounted Cumulative Gain

Because different queries have varying numbers of relevant documents, raw DCG_p scores cannot be compared across queries. We normalize by the **Ideal DCG** ($IDCG_p$), obtained by computing the DCG of the ideal ranking sorted by relevance descending:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (1.20)$$

1.8.3 Complete Numerical Walkthrough

Consider the following 10-document ranking, judged on a scale of 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

The ideal sorted ranking is:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

Table 1.4 summarizes the step-by-step progression across all 10 ranks.

i	r_i	Discount	Discounted Gain	$DCG@i$	r_i^*	Ideal Gain	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Table 1.4: Detailed trace of $DCG@i$, $IDCG@i$, and $NDCG@i$ computation for ranks 1 through 10.

1.8.4 Comparing Alternative Ranking Functions

Consider 4 documents $\{d_1, d_2, d_3, d_4\}$ with graded assessments:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordering $\langle d_4, d_3, d_2, d_1 \rangle$ with relevance vector $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordering $\langle d_3, d_4, d_2, d_1 \rangle$ with relevance $[2, 2, 1, 0]$. Because the first two documents share identical relevance ($r = 2$), their permutation incurs no positional penalty:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordering $\langle d_3, d_2, d_4, d_1 \rangle$ with relevance $[2, 1, 2, 0]$. Here, d_2 ($r = 1$) erroneously outranks d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + 1.2619 = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

This illustrates how NDCG mathematically penalizes rank inversions in direct proportion to the relevance difference.

Chapter 2

Algorithmic Efficiency on Modern CPU Architectures

2.1 The Role of Efficiency in Information Retrieval

The development of information processing systems unfolds along two complementary dimensions that are often in tension with one another:

1. **Effectiveness:** maximizes end-user satisfaction, focusing on the semantic quality of results, ranking, and natural language processing, typically using high-level abstraction tools (Python, Machine Learning frameworks).
2. **Efficiency:** minimizes infrastructural operating costs, designing algorithms optimized down to the last CPU clock cycle and the last bit of occupied memory.

No single professional figure is required to master both extremes; however, those who develop at a high level must understand the physical constraints of the underlying hardware to avoid designing impractical solutions, while those who optimize at a low level must preserve the structure and semantics of the data so as not to compromise its effectiveness.

2.1.1 The Three Pillars of Efficiency in IR Systems

User experience. In web-scale systems, a query response must be delivered within an infinitesimal fraction of a second. Users perceive any delay beyond a few tenths of a second: if results are slow or irrelevant, they abandon

the service in favor of a competitor. Algorithmic efficiency does not only serve to respond faster, but allows more sophisticated and effective ranking and filtering algorithms to run *within the same predetermined time budget* (typically a hard ceiling of 100–200 ms).

Operational scalability. Modern systems handle billions of documents, multimedia items, queries, and logs generated every second. The number of machines required is approximately:

$$\text{Number of machines} = \left\lceil \frac{\text{Total data volume}}{\text{Single-node operating capacity}} \right\rceil.$$

If the data volume doubles, a well-designed system requires at most twice as many machines (linear scalability); an inefficient algorithm can instead require superlinear cost growth (e.g., quadrupling servers), which is unsustainable at scale. To meet required latencies, active data must reside in RAM and avoid disk access: this requires algorithms that are efficient not only in time but also in space, via compression techniques. Furthermore, since data is replicated across multiple machines for fault tolerance, every memory inefficiency is amplified by the infrastructure’s replication factor.

Environmental and economic sustainability. Reducing the number of physical machines directly cuts data center energy consumption. High-density servers dissipate large amounts of heat, and much of the energy is consumed by cooling systems; for this reason many large-scale data centers are located in cold regions or near bodies of water (*free cooling*). A marginal computational saving, even just 3%, generates significant economic savings and reduces the carbon footprint.

2.2 The Memory Hierarchy and Locality Principles

2.2.1 The Latency Gap Between CPU and Memory

In the classical theoretical computational model (abstract RAM machine), access to any memory location has constant unit cost, $\mathcal{O}(1)$. On real physical machines, the CPU’s arithmetic units operate at speeds enormously higher than main memory (DRAM), and there exists a hierarchy of intermediate levels (*caches*) that approximates fast access for frequently used data.

Hardware level	Typical latency	CPU cycles	Typical capacity
CPU instruction (ALU)	0.5–5 ns	1–2	internal registers
L1I / L1D Cache	0.5 ns	≈ 1	32 KB per core
L2 Cache	1.5 ns	≈ 3 –4	256 KB (up to 2 MB)
L3 Cache (LLC)	5.0 ns	≈ 10 –15	16 MB (up to 64 MB+)
Main RAM (DRAM)	80.0 ns	≈ 160 –200+	64 GB+
Disk / Storage	500 000 ns (0.5 ms)	millions	8 TB+

Table 2.1: Quantitative comparison of latency and capacity across memory hierarchy levels.

Definition 2.2.1 (*Memory-Bound Algorithm*). An algorithm that accesses main RAM in a disordered fashion pays ≈ 80 ns per access, an interval during which the CPU could execute hundreds of instructions. Such an algorithm is called **memory-bound**: execution time is dominated by waiting for data transfer from DRAM, and the CPU spends most of its time stalled, making the optimization of arithmetic operation count irrelevant.

2.2.2 Physical On-Chip Architecture

The L1 cache is integrated close to the decode unit and execution units; to avoid bus conflicts it is physically split into **L1I** (*Instruction Cache*, holding fetched machine code for execution) and **L1D** (*Data Cache*, holding operands for **load/store** instructions). The L2 cache is integrated within the single core and acts as a low-latency buffer; the L3 cache is external to individual cores but internal to the package, and is shared among all cores of the machine.

The Cache Line as the Minimum Transfer Unit

RAM and the cache hierarchy never transfer single bytes or isolated individual variables: the minimum hardware transfer unit is the **cache line**, typically $B = 64$ bytes. If a program needs a 64-bit integer (8 bytes) located at address A , the hardware is forced to load the entire aligned block

$$[\lfloor A/64 \rfloor \times 64, \lfloor A/64 \rfloor \times 64 + 63].$$

If the program reads that single value and then jumps to a random address, the remaining 56 bytes (87.5% of the memory traffic) are wasted.

2.2.3 Temporal and Spatial Locality

A program effectively exploits the memory hierarchy only if it respects at least one of the two forms of locality:

- **Temporal locality:** when a datum accessed at time t is likely to be accessed again shortly thereafter (e.g., loop counters, frequently reused variables).
- **Spatial locality:** when accessing an address makes it likely that nearby addresses will be accessed soon after (e.g., sequential array traversal), which hardware exploits through the cache line and the **prefetcher** (a unit that detects linear access patterns and speculatively loads upcoming cache lines before they are explicitly requested).

Definition 2.2.2 (Sequential Access vs. *Pointer Chasing*). A **sequential** access pattern visits contiguous memory addresses in increasing order, without isolated loops or disjoint sub-cycles. **Pointer chasing** instead follows a chain of pointers whose target addresses are unpredictable in advance and typically scattered across memory, defeating spatial locality and prefetching.

Both functions below compute the sum of all elements of the vector, $\sum_{i=0}^{N-1} v[i] = \frac{(N-1)N}{2}$.

Listing 2.1: Linear sequential scan.

```
1 pub fn sum(v: &[usize]) -> usize {
2     let mut sum = 0;
3     for &value in v.iter() {
4         sum += value;
5     }
6     sum
7 }
```

Listing 2.2: Scan via random jumps (*pointer chasing*).

```
1 pub fn sum_by_jumping(v: &[usize]) -> usize {
2     let mut sum = 0;
3     let mut k = 0;
4     for _ in 0..v.len() {
5         sum += v[k];
6         k = v[k]; // the value read becomes the index
7                   for the next iteration
8     }
9     sum
10 }
```

In the theoretical RAM model both functions execute N iterations with one addition per step: they have identical asymptotic complexity $\mathcal{O}(N)$. On real hardware, measuring the average time per element as array size varies, marked divergences emerge instead.

N	Size	Cache level	Time per elem.		Slowdown
			sum()	jumping()	
256	2 KiB	L1 (32 KB)	0.1 ns	1.3 ns	13×
8 192	64 KiB	L2 (256 KB)	0.1 ns	2.5 ns	25×
262 144	2 MiB	L3 (16 MB)	0.2 ns	13.4 ns	67×
16 777 216	128 MiB	RAM (DRAM)	0.4 ns	84.9 ns	212×

Table 2.2: Average time per element for `sum()` and `sum_by_jumping()` as array size varies.

2.2.4 Analysis of Physical Causes

Why does `sum()` only vary between 0.1 and 0.4 ns? With 64-bit `usize` integers (8 bytes), a 64-byte cache line holds $64/8 = 8$ contiguous elements: there is full spatial locality. Accessing a new line incurs an initial miss, but the following 7 accesses occur at near-zero cost from cache, and the stream prefetcher loads subsequent lines ahead of execution. Denoting by T_{comp} the computation time and by T_{load} the amortized transfer time, the observed cost per element follows

$$\text{Observed latency} \approx \max(T_{\text{comp}}, T_{\text{amortized_load}}),$$

almost entirely masked by computation; the increase from 0.1 to 0.4 ns at 128 MiB is due solely to saturation of the bus bandwidth toward RAM.

Why does `sum_by_jumping()` reach 84.9 ns? Since the permutation is random, the address $k_{t+1} = v[k_t]$ points to an arbitrary location: the probability that it falls in the same cache line is negligible, so for every 8-byte element read the remaining 56 transferred bytes are discarded (total absence of spatial locality). With no linear or constant stride, prefetching is inhibited. Moreover, the CPU must know the index of the next read before it can issue it: this is a **serial dependency** that prevents any overlap. When the array size exceeds L3 capacity (128 MiB $\gg$ 16 MB), every access becomes a global cache miss to DRAM:

$$\text{Time per element} \approx \text{physical DRAM access latency} \approx 80\text{--}85 \text{ ns},$$

a value that matches exactly the 84.9 ns measured.

Example 2.2.1 (Amortizing the Cost of *Pointer Chasing* by Exploiting the Cache Line). If the jump occurs between 64-byte blocks (8 64-bit integers), but all elements of the block are summed before the next jump, the cost of a cache miss to RAM (≈ 85 ns) is paid only once every 8 addition operations:

```
1 pub fn sum_by_jumping_cache_lines(v: &[usize]) -> usize
2 {
3     let mut sum = 0;
4     let mut k = 0;
5     let num_blocks = v.len() / 8;
6     for _ in 0..num_blocks {
7         let base = k * 8;
8         // fully exploits the cache line already loaded
9         // into memory
10        sum += v[base] + v[base + 1] + v[base + 2] + v[
11            base + 3]
12            + v[base + 4] + v[base + 5] + v[base + 6];
13        k = v[base + 7] % num_blocks; // pointer to the
14        next jump
15    }
16    sum
17 }
```

The latency is amortized over 8 elements, bringing the expected average time per value to

$$\text{Average time per value} \approx \frac{85 \text{ ns}}{8} \approx 10.6 \text{ ns},$$

an intermediate result between linear scanning and element-by-element random jumping.

Remark 2.2.1 (Eliminating the Residual Slowdown of `sum()`). The slight slowdown from 0.1 to 0.4 ns observed in `sum()` occurs because $T_{\text{comp}} \approx 0.1$ ns is lower than the time needed to feed the data stream from RAM for very large vectors (memory-bound regime). By increasing the computational complexity of the loop body (e.g. multiplications or non-trivial functions on each element) one obtains $T_{\text{comp}} \gg T_{\text{load_RAM}}$: the relation $\max(T_{\text{comp}}, T_{\text{load}})$ becomes constantly equal to T_{comp} , the regime becomes *CPU-bound*, and the time curve becomes perfectly flat as N varies.

2.3 Forms of Parallelism and Pipeline Architecture

Modern CPUs exploit multiple forms of parallelism: **multi-threading/multi-core** (multiple independent physical cores on the same processor), **pipelining** (splitting instruction execution into stages operating simultaneously on adjacent instructions), and **superscalar** execution (multiple compute units within the same core to process several independent instructions per cycle).

Embarrassingly Parallel Workloads in Search Engines

Search engines do not need to synchronize multiple cores to answer a single query: they adopt **trivial parallelism** (*embarrassingly parallel*), in which, given thousands of simultaneous users, each thread on a single core serves a specific query in complete isolation. With no shared state, lock contention and cache coherence issues are eliminated.

2.3.1 Anatomy of the Five-Stage Pipeline

Pipelining decomposes instruction execution into independent sequential stages, which in a standard architecture are five:

1. **IF** (*Instruction Fetch*): fetching the machine-code instruction from memory (L1I) via the *program counter*.
2. **ID** (*Instruction Decode*): decoding the instruction into micro-operations (μ -ops) and reading source registers.
3. **EX** (*Execute*): performing the arithmetic-logic computation in the ALUs or computing the effective memory address.
4. **MEM**: access to the memory subsystem (L1D) for load or store instructions (LOAD/STORE).
5. **WB** (*Write-Back*): writing the result to architectural registers and retiring the instruction.

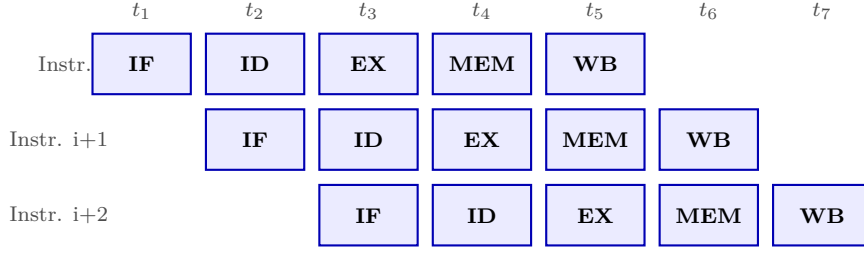


Figure 2.1: Overlap of the five pipeline stages across consecutive instructions.

2.3.2 Latency, Throughput, and Pipeline Depth

Let k be the number of pipeline stages and τ the clock cycle period. The **latency** L is the total time required for a single instruction to traverse the entire pipeline, $L = k \cdot \tau$: in modern CPUs the latency for a simple operation typically ranges between 10 and 20 clock cycles. The **throughput** W is the number of instructions completed per unit time, and at steady state $W = 1/\tau$ instructions per second, since one instruction exits the pipeline every cycle. The period τ is dictated by the delay of the slowest stage in the circuit chain,

$$\tau \geq \max_{1 \leq i \leq k} (t_{\text{stage}_i}).$$

To raise the clock frequency (reduce τ), designers increase pipeline depth k (up to 15–20 or more stages in modern CPUs), implementing each stage with simpler, faster circuitry; conversely, increasing k inflates the penalty in case of a forced pipeline flush.

2.4 Superscalar Execution, Interleaving, and Structural Hazards

A processor is called **superscalar** when it has the architectural ability to issue and complete more than one instruction per clock cycle ($IPC > 1$, *Instructions Per Cycle*) within the same core, integrating multiple arithmetic-logic units (integer ALUs, multipliers, shift units, load/store units) served by parallel issue ports (*execution ports*). Real architectures such as Intel Haswell feature, for instance, a 56-entry decode queue, a 192-entry reorder buffer, a unified 60-entry reservation station, and 8 parallel execution ports, each specialized for classes of operations (integer ALUs, load/store, vector units, branch, etc.).

2.4.1 The Arithmetic *Interleaving* Experiment

Four Rust functions operating on a vector of 64-bit integers `&[u64]` are benchmarked, accumulating an increasing number of independent operations per iteration.

Listing 2.3: From a single reduction to four independent reductions per iteration.

```
1 pub fn do_stuff(v: &[u64]) -> u64 {
2     let mut prod = 1;
3     for &value in v.iter() {
4         prod *= value;
5     }
6     prod
7 }
8
9 pub fn do_more_stuff(v: &[u64]) -> (u64, u64) {
10     let mut prod = 1;
11     let mut sum = 0;
12     for &value in v.iter() {
13         prod *= value;
14         sum += value;
15     }
16     (prod, sum)
17 }
18
19 pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {
20     let mut prod = 1;
21     let mut sum = 0;
22     let mut xor = 0;
23     for &value in v.iter() {
24         prod *= value;
25         sum += value;
26         xor ^= value;
27     }
28     (prod, sum, xor)
29 }
30
31 #[inline]
32 pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64) {
33     let mut prod = 1;
34     let mut sum = 0;
35     let mut xor = 0;
```

```

36     let mut or = 0;
37     for &value in v.iter() {
38         prod *= value;
39         sum += value;
40         xor ^= value;
41         or |= value;
42     }
43     (prod, sum, xor, or)
44 }

```

Function	Time per iteration
do_stuff() (1 operation)	0.4 ns
do_more_stuff() (2 operations)	0.4 ns
do_more_stuff_2() (3 operations)	0.4 ns
do_more_stuff_3() (4 operations)	0.5 ns

Table 2.3: Time per iteration as the number of accumulated independent reductions grows.

2.4.2 Instruction Overlap and Port Saturation

From one to three operations the time remains unchanged at 0.4 ns because the accumulator variables have no mutual dependencies: $\text{prod}_{t+1} = \text{prod}_t \cdot v[i]$, $\text{sum}_{t+1} = \text{sum}_t + v[i]$, $\text{xor}_{t+1} = \text{xor}_t \oplus v[i]$. The superscalar core simultaneously dispatches the multiplication, addition, and XOR to three distinct execution ports: the three operations complete in the same clock cycle, and the extra work is performed at zero cost.

With the fourth operation (`or`) the time rises to 0.5 ns rather than doubling to 0.8 ns: this is a **structural hazard**, since the physical units available for parallel arithmetic-logic computation in a single cycle are saturated (only three instructions of that type can be issued simultaneously), and one of the four operations must wait. However the *out-of-order* CPU does not process iterations as isolated compartments: it implicitly unrolls the instruction window, grouping them into blocks of three, absorbing the excess operation into subsequent cycles and saturating the residual capacity of the ports. The real overhead thus amounts to a marginal fraction of a cycle ($\Delta t = +0.1$ ns) rather than a full additional cycle.

2.5 Data Hazards and Serial Dependencies

Definition 2.5.1 (Data Hazard). A **data hazard** occurs when an instruction needs an operand that is not yet available because it is still being computed by a preceding instruction. It corresponds to a **RAW** (*Read-After-Write*) data dependency:

$$\text{Inst}_A : R_1 \leftarrow R_2 + R_3 \quad \Longrightarrow \quad \text{Inst}_B : R_4 \leftarrow R_1 \times R_5,$$

where Inst_B must stall in the pipeline until Inst_A completes its computation.

Consider code that performs the same number of operations per iteration as the previous `do_more_stuff_3`, but chains the data in a tight dependency:

Listing 2.4: Serial RAW dependency chain across four operations.

```

1 pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         sum += value; // 1. compute sum
7         prod *= sum; // 2. tightly depends on sum
8         xor ^= prod; // 3. tightly depends on prod
9         sum += xor; // 4. depends on xor and
10                // overwrites sum for the next iteration!
11     }
12     (prod, sum, xor)
13 }
```

Because of the RAW chain, the execution units cannot start any computation in parallel:

$$\text{Iteration latency} = t_{\text{add}} + t_{\text{mul}} + t_{\text{xor}} + t_{\text{add_feedback}}.$$

Since the last operation updates `sum` and the first instruction of the next iteration requires the new value of `sum`, the *out-of-order* CPU cannot start iteration $t + 1$ early: the superscalar units remain unused. Experimentally, against the 0.5 ns of `do_more_stuff_3` (independent operations), `do_more_stuff_4` measures 1.7 ns per iteration ($\approx 3.5\times$ slower), despite performing the same number of arithmetic operations.

2.5.1 Algorithmic Solution: *Interleaving* Independent Vectors

When running this computation on two distinct vectors `V` and `West`, the sequential approach (calling `do_more_stuff_4` on `V` and then on `West`) requires

total time $2T$. If instead a single function processes both vectors within the same iteration, alternating the two independent chains:

Listing 2.5: Fusion (interleaving) of two mutually independent RAW chains.

```

1  for i in 0..n {
2      // chain for V (occupies the ports at cycle k)
3      sum_v += v[i];
4      prod_v *= sum_v;
5      xor_v ^= prod_v;
6      sum_v += xor_v;
7      // chain for West, completely independent of V!
8      sum_w += west[i];
9      prod_w *= sum_w;
10     xor_w ^= prod_w;
11     sum_w += xor_w;
12 }
```

the measured execution time is identical to that of the single vector *V* alone ($\approx T$, rather than $2T$). Operations on *West* do not depend on those on *V*: while *V*'s chain is waiting for ALU results, the free compute units process *West*'s chain, filling the dead time imposed by data dependencies at zero extra cost.

2.6 Control Hazards, Speculative Execution, and Branch Prediction

Definition 2.6.1 (Control Hazard). A **control hazard** occurs when the CPU cannot determine which instruction to fetch next (*instruction fetch*) due to the presence of a conditional branch (*if*, *branch*).

The CPU cannot stop at every condition: it therefore adopts **speculative execution**, guessing via a *branch predictor* which branch will be taken and speculatively fetching subsequent instructions to feed the pipeline. If the outcome of the condition turns out to disagree with the prediction, all speculative work completed in the intermediate stages must be discarded, with a full pipeline flush and reload from the correct branch:

Misprediction penalty = 15–20 clock cycles.

2.6.1 The Random Condition Paradox

A conditional filter is introduced into code with a dependency chain, operating on an array of random integers uniformly distributed in $[0, 99]$:

Listing 2.6: Conditional branch on random values: the CPU cannot predict the outcome.

```
1 pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         if value > 50 { // true with p = 0.5 on random
7             // values in [0, 99]
8             sum += value;
9             prod *= sum;
10            xor ^= prod;
11            sum += xor;
12        }
13    }
14    (prod, sum, xor)
}
```

Since the condition is true on average 50% of the time, one would intuitively expect a time roughly halved compared to `do_more_stuff_4` ($1.7/2 \approx 0.85$ ns). The measured time instead rises to 3.5 ns, more than twice as slow as the version without the condition.

The Real Cost Is Not Computation but Misprediction

With random values the branch predictor's accuracy is equivalent to a coin flip, $P(\text{hit}) = P(\text{miss}) = 0.50$. The average cost per iteration is then:

$$\begin{aligned}\text{Average cost} &\approx 0.5 \times T_{\text{body}} + P(\text{miss}) \times \text{Flush penalty} \\ &\approx 0.5 \times 0.85 \text{ ns} + 0.5 \times 5.0 \text{ ns} \approx 2.9\text{--}3.5 \text{ ns},\end{aligned}$$

having assumed a penalty of 15–20 cycles ($\approx 4\text{--}5$ ns on a 4 GHz CPU). The cost of the algorithm is no longer dictated by arithmetic operations, but is dominated by the time lost flushing and refilling the pipeline.

2.6.2 Experimental Verification: Data Ordering and Periodic Patterns

To demonstrate that the slowdown derives exclusively from branch predictor behavior, the same identical code is run while only changing the arrangement of data in the array.

Configuration	Time / iter.	Branch predictor behavior
do_more_stuff_4() (no if)	1.7 ns	—
do_more_stuff_5() (random data)	3.5 ns	Unpredictable branch, miss rate $\approx 50\%$
do_more_stuff_5() (sorted data)	1.2 ns	Near-perfect prediction, single miss
do_more_stuff_5() (alternating data)	1.1 ns	Periodic sequence, miss rate $\approx 0\%$

Table 2.4: Time per iteration of `do_more_stuff_5()` as data arrangement varies.

With **sorted data** (all elements ≤ 50 at the start of the array and all elements > 50 at the end) the predictor assumes the branch is not taken for the whole first half and taken for the second: a single misprediction occurs at the transition point, and once flush costs are removed the time drops to 1.2 ns, lower than the 1.7 ns of the version without the condition since half the iterations do not execute the `if` body. With **alternating data** following a periodic pattern (*false, true, false, true, ...*), predictors integrate historical pattern tables (*branch history shift register*): once the periodicity is identified, the predictor anticipates the outcome at every step without errors, eliminating stalls.

2.6.3 *Branchless* Techniques and the Implicit Jump of `for` Loops

If data cannot be sorted or predicted, the only solution is to physically remove the conditional branch (*branchless code*), replacing it with algebraic expressions or conditional assembly instructions (`CMOV`). For example, instead of

```

1 if value > 50 {
2     sum += value;
3 }

```

the logical guard is mapped onto a scalar binary value (0 or 1), always performing the arithmetic operation:

```

1 let condition = (value > 50) as u64; // 1 if true, 0 if
   false
2 sum += value * condition;           // adds 'value' if
   true, adds '0' if false

```

The program always performs the mathematical computation, but the instruction flow is linear: there are no branches, and mispredictions are eliminated.

Remark 2.6.1 (Why the `for` Loop Branch Is Not a Problem). Every `for` loop ends at the machine level with a conditional jump back to the start of the block. This branch does not degrade performance because it is highly asymmetric: for N iterations, the loop-continuation condition is true $N - 1$ consecutive times and false only upon exit. The branch predictor immediately learns the pattern (“branch taken”) and commits a single misprediction at the end of the loop, with a completely negligible impact on overall time.

2.7 Synthesis: From Hardware Constraints to Algorithmic Choices

The phenomena analyzed in this chapter – memory hierarchy and locality, pipelining, superscalar execution, *data hazards*, and *control hazards* – are not negligible implementation details, but physical constraints that determine the actual performance behavior of large-scale Information Retrieval algorithms. In particular:

- data structures (posting lists, inverted indexes) should be designed to favor sequential, contiguous access, in order to exploit spatial locality and hardware prefetching;
- when possible, it is beneficial to express multiple independent reductions within the same loop to saturate superscalar execution ports at marginal cost;
- tight (RAW) dependency chains between accumulator variables should be identified and, where possible, decoupled or interleaved with independent computations from other data streams;
- unpredictable conditional branches on data-dependent conditions should be replaced with branchless techniques whenever the misprediction cost outweighs the extra computation.

Chapter 3

Natural Language Processing and Text Representation

3.1 Graded Relevance Metrics (CG, DCG, NDCG)

In real-world Information Retrieval systems, the assumption of purely binary relevance (relevant vs. non-relevant) is inadequate to capture the nuances of search results. Relevance judgments are therefore expressed on graded scales (e.g., a Likert scale from 0 to 3 or 0 to 4, indicating increasing degrees of pertinence).

The evaluation of a ranked list of documents up to position p is founded on two core principles:

1. **Gain:** The higher a document with high relevance appears within the SERP (*Search Engine Results Page*), the greater the utility perceived by the user.
2. **Discounting:** Highly relevant documents placed at lower ranks deliver progressively less utility than if placed at the top, since user attention monotonically decays down the list.

3.1.1 Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)

Given a vector of graded relevance scores $\mathbf{r} = [r_1, r_2, \dots, r_p]$ for the top- p documents returned for a query q :

Definition 3.1.1 (Cumulative Gain at Rank p). **Cumulative Gain** (CG_p) is the unweighted sum of relevance values up to rank p :

$$CG_p = \sum_{i=1}^p r_i. \quad (3.1)$$

Inherent limitation: CG_p is position-insensitive: placing a highly relevant document ($r_i = 3$) at rank 1 or rank 10 produces identical CG scores.

To model user attention decay as rank i increases, a logarithmic discount function is applied:

Definition 3.1.2 (Discounted Cumulative Gain at Rank p). In the standard formulation and the exponential gain variant widely adopted in industrial search engines:

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)}, \quad (3.2)$$

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)}. \quad (3.3)$$

The exponential variant (3.3) heavily penalizes the failure to place top-relevance documents at the very highest ranks.

3.1.2 Normalized DCG (NDCG)

Raw DCG has a critical structural drawback: it is strictly **query-dependent**. If a specific query q_A has 100 highly relevant documents ($r = 3$) in the collection, its theoretical DCG will naturally be huge. If another query q_B has only one relevant document in the entire corpus, its maximum attainable DCG will be very small. Consequently, raw DCG scores cannot be directly compared or averaged across different queries.

To enable cross-query comparison and global macro-averaging across an evaluation benchmark, we compute the **Ideal DCG** ($IDCG_p$), which is the DCG of the ideal ranking obtained by sorting all documents in descending order of relevance:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1]. \quad (3.4)$$

With normalization, $NDCG_p = 1.0$ guarantees that the search engine achieved the best possible ordering up to depth p .

3.2 Characteristics of Natural Language and Zipf's Law

3.2.1 Properties and Inherent Complexity of Natural Language

Processing natural language computationally presents unique hurdles due to its structural characteristics:

- **Vast vocabularies:** Real-world vocabularies contain hundreds of thousands of distinct words.
- **Complex, hierarchical syntax:** Sentences are not mere flat sequences, but nested syntax trees with long-range grammatical dependencies.
- **Compositional semantics:** The overall meaning of an utterance is a function of the meanings of its parts and their syntactic arrangement.
- **Pervasive ambiguity:**
 - *Part-of-Speech (POS) ambiguity:* A single word form functions as noun or verb depending on context (e.g., *beat*: “*Listen to this cool beat*” [noun] vs. “*I’ll beat you at checkers!*” [verb]).
 - *Word sense ambiguity (Polysemy and Homonymy):* Multiple distinct meanings (e.g., *interest*: “*There is interest on this topic*” [curiosity] vs. “*The bank raised the interest rate*” [financial charge]).
 - *Syntactic ambiguity:* Sentences with multiple valid parse trees (e.g., “*I shot an elephant in my pajamas*”).
- **Heavy reliance on Shared World Knowledge:**
 - “*I’m faster than Lewis Hamilton*”: Requires knowing Lewis Hamilton is a Formula 1 driver to interpret what speed is being compared.
 - “*Alexis and Kate are sisters*”: Asserts a symmetric kinship relation between two entities.
 - “*Alexis and Kate are mothers*”: Attributes a shared ontological property to both individuals, without implying a direct relationship between them.
 - “*My mother is younger than me*”: Syntactically well-formed, but semantically and biologically impossible.

- **Temporal evolution:** Vocabularies evolve rapidly through technological innovation and socio-cultural shifts (e.g., “*to google*”, “*crowdfunded*”, “*in the cloud*”, “*super spreader*”, “*wear a mask*”).

3.2.2 Zipf’s Law and the Principle of Least Effort

In any natural language corpus, the empirical frequency f of a term is inversely proportional to its rank r when sorted by frequency:

$$f \propto \frac{1}{r} \implies f \cdot r \approx C, \quad (3.5)$$

where C is a corpus-specific constant. In Mandelbrot’s generalized analytical formulation (*Zipf-Mandelbrot law*):

$$f(r) \propto \frac{1}{(r + \beta)^\alpha}, \quad (3.6)$$

typically with $\alpha \approx 1$ and $\beta \approx 2.7$.

Zipf’s Principle of Least Effort

George Kingsley Zipf explained this hyperbolic distribution as an equilibrium between two opposing cognitive forces:

1. **Speaker’s effort:** Aims to minimize cognitive strain by using a small vocabulary of frequent, short, polysemous words.
2. **Hearer’s effort:** Prefers a large vocabulary of distinct, rare, and precise words to minimize decoding ambiguity.

The evolutionary compromise between speaker economy and listener clarity yields Zipf’s law.

From Zipf’s formulation follow two key linguistic properties:

- The number of meanings m of a word scales inversely with the square root of its frequency: $m \propto 1/\sqrt{f}$.
- An inverse relationship connects word frequency and word length: high-frequency words are almost always the shortest.

3.2.3 Rare Features and Stop Words

Zipf’s distribution exposes two critical extremes for search system design:

1. **Rare Features and *Hapax Legomena*:** Words appearing only once in the entire corpus (*hapax legomena*) typically comprise 40% to 50% of vocabulary $|V|$. A feature may be rare for two distinct reasons:
 - It is a highly specialized technical term with **enormous discriminative content** (e.g., “*arachnocentric*”).
 - It is a spelling typo or an arbitrary URL/file slug.

Pruning rare terms drastically shrinks index size, but risks eliminating highly specific search needles.

2. **Stop Words:** Extremely high-frequency syntactic glue words (“*the*”, “*of*”, “*a*”, “*to*”, “*in*”). Historically stripped during indexing to save memory and boost traversal speed (“*the president of the united states of america*” → “*president united states america*”).

Warning on Stop Word Lists

Stop word lists are domain-specific. For instance, MySQL’s default stop word list originally included words like “*appreciate*”, “*serious*”, and “*unfortunately*”, which are vital features for Sentiment Analysis and opinion-oriented search tasks.

3.3 Data Acquisition and Web Parsing: urllib and BeautifulSoup

The initial step in search engine construction is crawling and HTML text extraction, depicted in Figure 3.1.

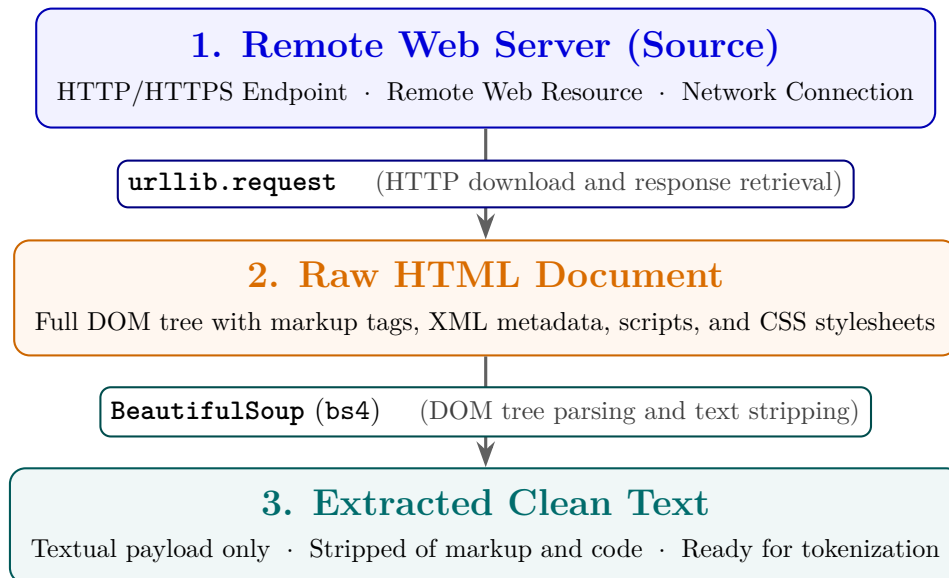


Figure 3.1: HTTP web crawling and HTML stripping pipeline.

3.3.1 Acquisition with `urllib`

In Python, low-level HTTP requests are handled cleanly via `urllib.request`:

Listing 3.1: Fetching raw HTML payload over HTTP.

```
1 from urllib import request
2
3 url = "http://hpc.isti.cnr.it/~nardini/"
4 response = request.urlopen(url)
5 html = response.read().decode('utf8')
6 # Yields the full HTML document string with markup
```

3.3.2 DOM Parsing and Stripping with BeautifulSoup

Web pages embed scripts and formatting markup that would pollute term statistics. BeautifulSoup navigates the DOM tree to extract pure content:

Listing 3.2: HTML markup stripping with BeautifulSoup.

```
1 from bs4 import BeautifulSoup
2
3 soup = BeautifulSoup(html, "html5lib")
4
5 # Direct extraction of all concatenated text:
```

```

6 text_all = soup.get_text()
7
8 # Granular extraction targeting only paragraph (<p>)
  tags:
9 text_paragraphs = [''.join(s.findAll(text=True)) for s
  in soup.findAll('p')]

```

3.4 Text Preprocessing Pipeline: Tokenization and Sentence Splitting

Data Engineering Rule of Thumb

In real-world IR and NLP architectures, **80% of engineering time and effort** is spent on data ingestion, cleaning, normalization, and preprocessing pipelines before any model or indexing algorithm can be run.

3.4.1 Tokenization

Tokenization breaks raw character streams into discrete linguistic units called **tokens**:

Listing 3.3: Tokenization and stop word removal using NLTK.

```

1 import nltk
2 from nltk import word_tokenize
3 from nltk.corpus import stopwords
4
5 text = "the president of the united states of america"
6 tokens = word_tokenize(text)
7 # Output: ['the', 'president', 'of', 'the', 'united', 'states', 'of', 'america']
8
9 stop_words = set(stopwords.words('english'))
10 features = set(tokens)
11 filtered_features = features.difference(stop_words)
12 # Output: {'america', 'president', 'states', 'united'}

```

3.4.2 Sentence Splitting and Punctuation Ambiguity

Segmenting unpunctuated or continuous text into standalone sentences (*sentence boundary disambiguation*) is challenging due to the overloaded role of the period (.):

1. Sentence terminator.
2. Acronym, abbreviation, or honorific marker (*U.S.A.*, *e.g.*, *St. Louis, Dr.*).
3. Numerical decimal or time separator (*8.30*).
4. Discourse ellipsis (...).

Example 3.4.1 (Sentence Boundary Disambiguation). Consider the classroom example:

“I’ll take the 8.30 train to St. Louis... or maybe I’ll stay in LA. Time will tell!”

A naive `split(' .')` rule produces severe errors:

- The dot in *8.30* is part of a time token.
- The dot in *St.* marks an abbreviation for *Saint*.
- The ellipsis (...) indicates a rhetorical pause without closing the logical clause.
- Only the dot after *LA.* and the exclamation point in *Time will tell!* are true sentence boundaries.

NLTK’s `sent_tokenize` employs the unsupervised *Punkt* tokenizer, trained to identify collocations and abbreviations to preserve genuine sentence boundaries.

3.5 Vocabulary Construction and Counting

3.5.1 Formal Vocabulary Definition

Definition 3.5.1 (Collection Vocabulary). Given a document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$, the **Vocabulary** (or feature set, denoted by V or F) is the union of all distinct term types across all documents:

$$V = \bigcup_{d \in \mathcal{D}} \{t \in d\}. \quad (3.7)$$

Its cardinality $|V|$ defines the dimensionality of the vector feature space. It is strictly **dataset-dependent**: adding or removing documents alters V and its dimension.

3.5.2 Algorithmic Analysis: Choosing the Counting Data Structure

When designing term frequency counters, beginners often suggest using a set (**set**):

- **Why a set fails:** A set enforces element uniqueness and discards duplicates upon first encounter, making it impossible to accumulate occurrence counts.
- **Correct approach:** A hash table (in Python, **dict** or **Counter**), mapping each unique token string to its running integer count.

Algorithm 1 details this procedure.

Algorithm 1: Vocabulary Construction and Collection Term Frequency Counting

Input: Document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$
Output: Vocabulary V and frequency map `freq_dict`

```

1 freq_dict  $\leftarrow$  empty hash map
2 foreach  $d \in \mathcal{D}$  do
3   tokens  $\leftarrow$  WordTokenize(d)
4   foreach  $t \in \text{tokens}$  do
5     if  $t \in \text{freq\_dict}$  then
6       freq_dict[t]  $\leftarrow$  freq_dict[t] + 1
7     else
8       freq_dict[t]  $\leftarrow$  1
9  $V \leftarrow \text{keys}(\text{freq\_dict})$ 
10 return  $V, \text{freq\_dict}$ 

```

At completion, $|V| = \text{len}(\text{freq_dict})$, and the collection frequency cf_t is recorded for every term $t \in V$.

3.6 Morphological Normalization: Stemming vs. Lemmatization

To collapse inflected variants into a single canonical index form, two competing paradigms exist (summarized in Table 3.1).

Criterion	Stemming	Lemmatization
Goal	Strip affixes to reach a crude root (<i>stem</i>).	Map word form to legitimate dictionary base (<i>lemma</i>).
Methodology	Rule-based heuristic suffix chopping.	Complete morphological and syntactic analysis via POS and lexical graphs.
Lexical Validity	Low: often generates non-words or invalid stems.	High: always produces genuine dictionary words.
POS Dependency	None: operates purely on character surface strings.	Crucial: requires contextual Part-of-Speech tagging.
Computational Cost	Negligible: instantaneous string rewrites in $\mathcal{O}(1)$.	Heavy: requires statistical sequence models or graph lookups.

Table 3.1: Systematic comparison between heuristic Stemming and Lemmatization.

3.6.1 Stemming and the Porter Stemmer Failure Case

The Porter Stemmer applies sequential suffix-stripping rules (e.g., -ing, -s, -ed).

Listing 3.4: Stemming with PorterStemmer illustrating lexical breakdown.

```

1 from nltk.stem.porter import PorterStemmer
2
3 stemmer = PorterStemmer()
4 stemmer.stem('cars') # Result: 'car' (valid)
5 stemmer.stem('was') # Result: 'wa' (LEXICAL ERROR)

```

Root Cause of the Stemmer Error on “was”

The stemmer identifies the trailing 's' in “was” and mechanically applies the English plural reduction rule. Lacking syntax awareness (that “was” is an inflected past tense form of *to be*), it produces the nonexistent pseudo-stem 'wa'.

3.6.2 Lemmatization with WordNet

Lemmatization queries a lexical ontology (WordNet). Without an explicit POS tag, the default assumption is that the token is a noun:

Listing 3.5: Context-aware lemmatization with WordNetLemmatizer.

```
1 from nltk.stem.wordnet import WordNetLemmatizer
2
3 lmtzr = WordNetLemmatizer()
4 lmtzr.lemmatize('cars')           # Output: 'car'
5 lmtzr.lemmatize('was')           # Output: 'wa'
   (default: noun)
6 lmtzr.lemmatize('was', pos='v')   # Output: 'be'
   (CORRECT: verb 'to be')
```

3.6.3 Engineering Trade-offs in Large-Scale Search Engines

Historically, commercial search engines processing billions of web documents have heavily favored **Stemming**: its near-zero computational overhead outweighs occasional morphological errors when balancing indexing throughput across planetary datasets.

3.7 The Bag of Words (BoW) Model

3.7.1 Transition from Sequence to Set

Unstructured text originates as an ordered token sequence:

$$\text{text} \longrightarrow [t_1, t_2, t_3, \dots, t_k]. \quad (3.8)$$

The **Bag of Words (BoW)** model abstracts the text by collapsing this sequence into an unordered set (or multiset with counts):

Listing 3.6: Bag of Words set representation.

```
1 text = 'the president of the united states of america'
2 tokens = word_tokenize(text)      # Length 8 list
3 bow = set(tokens)                 # 6 unique items
4 # bow: {'america', 'of', 'president', 'states', 'the', 'united'}
```

3.7.2 Structural Flaws of BoW

1. **Loss of local term frequencies** in pure set formulations.
2. **Total loss of word order and syntax:** BoW assumes word placement is irrelevant to semantic content.

Example 3.7.1 (Semantic Collapse in Bag of Words). Consider two sentences with diametrically opposed meanings:

- t_1 : “*I won, and thus you lose.*”
- t_2 : “*I lose, and thus you won.*”

Evaluating their BoW representations in Python:

Listing 3.7: Demonstration of BoW semantic indistinguishability.

```
1 t1 = set(word_tokenize('I won, and thus you lose.'))
2 t2 = set(word_tokenize('I lose, and thus you won.'))
3
4 print(t1 == t2) # Output: True
```

Under BoW assumptions, the two texts are identical. This demonstrates that human language semantics is deeply **compositional** and sensitive to syntax order.

3.8 N-grams: Word n-grams and Character n-grams

To recover local context without building complete parse trees, the vocabulary is augmented with **n-grams**.

3.8.1 Word n-grams

A **word n-gram** is a contiguous sequence of n words, capturing local phrase order:

Listing 3.8: Local syntax preservation using word bigrams.

```
1 import nltk
2
3 t1 = set(nltk.ngrams(nltk.word_tokenize('I won, and thus
    , you lose.'), 2))
```

```

4 t2 = set(nltk.ngrams(nltk.word_tokenize('I lose, and
      thus, you won.'), 2))
5
6 # t1 contains: ('won', 'and'), ('you', 'lose'), ...
7 # t2 contains: ('lose', 'and'), ('you', 'won'), ...
8 print(t1 == t2) # Output: False

```

Bigrams ($n = 2$) preserve local subject-verb combinations, successfully telling t_1 and t_2 apart.

3.8.2 Character n-grams

Character n-grams decompose words into contiguous character slices of length n (commonly trigrams, $n = 3$). This provides resilience against typos and captures morphological affinity:

Listing 3.9: Typo tolerance using character trigrams.

```

1 t1 = set(nltk.ngrams('rainbow', 3))
2 t2 = set(nltk.ngrams('rainbaw', 3)) # Contains typo: 'a
   ' instead of 'o'
3
4 shared = t1.intersection(t2)
5 # Output: {('r', 'a', 'i'), ('a', 'i', 'n'), ('i', 'n',
   'b')}

```

Despite the misspelling, the two strings share 3 out of 5 trigrams (60%), allowing the search engine to detect high geometric proximity and trigger query auto-correction.

3.9 The Vector Space Model (VSM)

In the **Vector Space Model** (VSM), documents and queries are embedded into a real vector space whose dimension matches vocabulary size:

$$\text{Dimensionality} = |F| = |V|. \quad (3.9)$$

Every orthogonal axis corresponds to a distinct term $f \in V$. Documents and queries are vectors:

$$\vec{v}(d) \in \mathbb{R}^{|V|}, \quad \vec{v}(q) \in \mathbb{R}^{|V|}. \quad (3.10)$$

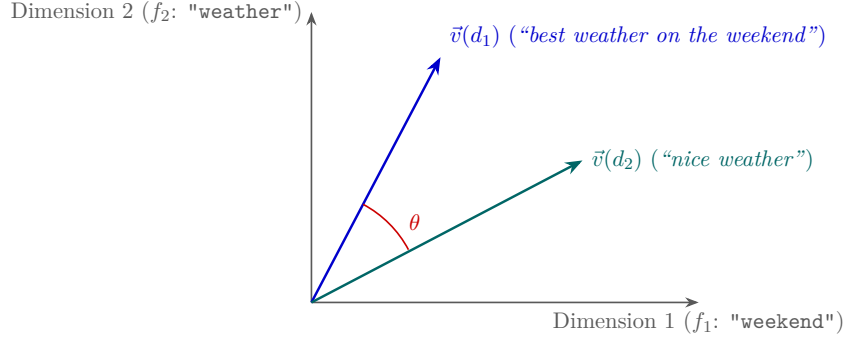


Figure 3.2: Two-dimensional geometric document vectors in the Vector Space Model.

3.9.1 Formal Mapping via Canonical Unit Vectors

Each feature $f_i \in V$ corresponds to a standard basis vector:

$$\vec{v}(d) = \sum_{f \in d} w_{f,d} \vec{v}(f), \quad (3.11)$$

where $w_{f,d} \in \mathbb{R}$ is the weight of term f in document d .

3.9.2 Extreme Vector Sparsity

For a typical web index with $|V| = 10^6$ and an average document length of 200 distinct words:

$$\frac{|\{i \mid v_i(d) \neq 0\}|}{|V|} = \frac{200}{1\,000\,000} = 0.02\%. \quad (3.12)$$

Over **99.98%** of elements are zeros. Storing them as dense 32-bit floating-point arrays (4 MB per document) would crash system memory, necessitating specialized sparse vector formats and inverted indices.

Chapter 4

Retrieval Models and Ranking: Vector Space Model and BM25

This chapter formalizes the fundamental Information Retrieval scoring models designed to compute continuous relevance scores (*ranking*) for ordering documents with respect to user queries. Moving beyond the limitations and structural pathologies of exact Boolean retrieval, we evaluate set-based matching with the Jaccard coefficient, develop the statistical foundations of TF-IDF term weighting and SMART notation within the Vector Space Model (VSM), and examine the modern probabilistic Okapi BM25 retrieval function alongside an end-to-end pipeline architecture.

4.1 Retrieval Models: Boolean vs Ranked Retrieval

Property	Boolean Retrieval	Ranked Retrieval
Match criteria	Exact logical satisfaction (in/out).	Continuous score $\text{Score}(q, d) \in \mathbb{R}$.
Query syntax	Strict operators (AND, OR, NOT).	Free text natural language.
Output format	Unordered document set.	Ordered top- k ranked list.
User experience	Feast or famine pathology.	Smooth, calibrated result presentation.

Table 4.1: Conceptual contrast between exact Boolean retrieval and ranked retrieval.

4.1.1 The “Feast or Famine” Pathology of Boolean Retrieval

- **Famine:** If a query is overly specific (too many AND clauses), no documents meet all criteria, yielding zero results.
- **Feast:** If the query is broad or uses OR, thousands of documents are returned with no relevance order.

Ranked retrieval eliminates this cognitive burden by scoring every document continuously and displaying only the top- k highest-scoring candidates.

4.2 Jaccard Similarity and its Limitations in Scoring

4.2.1 Formal Definition

Definition 4.2.1 (Jaccard Similarity Coefficient). Given two term sets A and B :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (4.1)$$

Properties: $J(A, A) = 1$, $J(A, B) = 0 \iff A \cap B = \emptyset$, and $0 \leq J(A, B) \leq 1$.

4.2.2 Worked Example and the Failure of Jaccard

Example 4.2.1 (Jaccard Paradox on Real Queries). Consider query q and two documents:

- **Query q :** “*ides of march*” with $A = \{\text{ides, of, march}\}$, $|A| = 3$.
- **Doc 1 (d_1):** “*caesar died in march*” with $B_1 = \{\text{caesar, died, in, march}\}$, $|B_1| = 4$.
- **Doc 2 (d_2):** “*the long march*” with $B_2 = \{\text{the, long, march}\}$, $|B_2| = 3$.

Computing Jaccard coefficients:

$$J(q, d_1) = \frac{1}{6} \approx 0.166,$$

$$J(q, d_2) = \frac{1}{5} = 0.200.$$

Paradoxical outcome: d_2 (“*the long march*”) scores higher than d_1 (“*caesar died in march*”) simply because d_2 is one word shorter, completely ignoring historical context.

4.2.3 Why Jaccard Fails for IR

1. **Ignores Term Frequency (tf):** Binary presence/absence treats 10 occurrences identically to 1.
2. **Ignores Term Informativeness (idf):** Common stop words like “of” are weighted equally with rare, content-rich words like “ides”.
3. **Unfair length penalty:** The denominator $|A \cup B|$ severely penalizes longer, richer documents.

4.3 Term Weighting: Term-Document Matrices and TF-IDF

4.3.1 Logarithmic Term Frequency

Relevance does not grow linearly with term occurrences. To model diminishing marginal utility, logarithmic damping is applied:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{if } tf_{t,d} > 0, \\ 0 & \text{if } tf_{t,d} = 0. \end{cases} \quad (4.2)$$

From Theoretical Matrix to Physical Data Structures

The term–document matrix is a theoretical construct. Its direct dense storage is intractable due to extreme sparsity ($> 99.99\%$). The physical transformation of this matrix into high-performance data structures (*Forward Index* and *Inverted Index*) and integer compression techniques are formalized in Chapter 5.

4.3.2 Document Frequency (df_t) vs. Collection Frequency (cf_t)

- **Collection Frequency (cf_t):** Total occurrences of term t across the entire collection.
- **Document Frequency (df_t):** Number of distinct documents containing t at least once ($df_t \leq N$).

In large corpora, “*Insurance*” ($cf = 10\,440$, $df = 3\,997$) and “*Try*” ($cf = 10\,422$, $df = 8\,760$) have identical collection frequencies, but “*Insurance*” occurs in less than half the documents, making it a much stronger topical discriminator.

4.3.3 Inverse Document Frequency (IDF)

$$\text{idf}_t = \log_{10} \left(\frac{N}{df_t} \right). \quad (4.3)$$

IDF depends strictly on the collection and is **independent of the query**, allowing offline precomputation. In single-term queries, IDF acts as a scalar multiplier with zero ranking impact. In multi-term queries, it balances specific terms against generic ones.

The combined **TF-IDF** weight is:

$$w_{t,d} = (1 + \log_{10} \text{tf}_{t,d}) \cdot \log_{10} \left(\frac{N}{df_t} \right). \quad (4.4)$$

4.4 Cosine Similarity and Length Normalization

4.4.1 Failure of Euclidean Distance

If document d is concatenated with itself to form $d' = d \circ d$, both treat the exact same topic, but $\|\vec{v}(d')\|_2 \approx 2\|\vec{v}(d)\|_2$. The Euclidean distance between them is massive despite identical semantics.

4.4.2 Cosine Similarity

Because $\vec{v}(d)$ and $\vec{v}(d')$ point in the exact same direction, the angle between them is $\theta = 0^\circ$. Minimizing θ is equivalent to maximizing $\cos(\theta)$:

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|_2 \|\vec{d}\|_2} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}. \quad (4.5)$$

Under L_2 vector normalization, this reduces to the inner product $\vec{q} \cdot \vec{d}$.

4.4.3 Length Normalization L_2

Dividing a vector by its Euclidean norm $\|\vec{x}\|_2$ projects it onto the unit hypersphere ($\|\vec{x}'\|_2 = 1$). For normalized vectors, cosine similarity simplifies to the dot product $\vec{q} \cdot \vec{d}$.

4.4.4 Worked Example: Comparing Three Classic Novels

Comparing Jane Austen’s *Sense and Sensibility* (SaS) and *Pride and Prejudice* (PaP) against Emily Brontë’s *Wuthering Heights* (WH):

$$\begin{aligned}\|\vec{v}(\text{SaS})\|_2 &= \sqrt{3.06^2 + 2.00^2 + 1.30^2} \approx 3.88, \\ \|\vec{v}(\text{PaP})\|_2 &= \sqrt{2.76^2 + 1.85^2} \approx 3.32, \\ \|\vec{v}(\text{WH})\|_2 &= \sqrt{2.30^2 + 2.04^2 + 1.78^2 + 2.58^2} \approx 4.39.\end{aligned}$$

Cosine similarities:

$$\begin{aligned}\cos(\text{SaS}, \text{PaP}) &\approx (0.789 \cdot 0.832) + (0.515 \cdot 0.555) \approx \mathbf{0.94}, \\ \cos(\text{SaS}, \text{WH}) &\approx (0.789 \cdot 0.524) + (0.515 \cdot 0.465) + (0.335 \cdot 0.405) \approx \mathbf{0.79}, \\ \cos(\text{PaP}, \text{WH}) &\approx (0.832 \cdot 0.524) + (0.555 \cdot 0.465) \approx \mathbf{0.69}.\end{aligned}$$

$$\cos(\text{SaS}, \text{PaP}) > \cos(\text{SaS}, \text{WH}) > \cos(\text{PaP}, \text{WH}), \quad (4.6)$$

namely $0.94 > 0.79 > 0.69$. The model correctly identifies high stylistic and thematic alignment between the two Austen novels while separating Brontë’s work.

4.5 SMART Notation and TF-IDF Variants

SMART notation standardizes weighting combinations in the form:

$$\mathbf{ddd.qqq}, \quad (4.7)$$

where the first triplet configures the document and the second configures the query.

4.5.1 Standard **1nc.1tc** Scheme

- **Document (1nc):** Log-TF, no IDF, cosine normalization.
- **Query (1tc):** Log-TF, standard IDF ($\log_{10}(N/df_t)$), cosine normalization.

For query “*best car insurance*” and document “*car insurance auto insurance*” ($N = 10^6$):

$$\begin{aligned}\|\vec{q}\|_2 &\approx 3.83, \quad \|\vec{d}\|_2 \approx 1.92, \\ \text{Score}(q, d) &= (0.52 \cdot 0.52) + (0.78 \cdot 0.68) = \mathbf{0.80}.\end{aligned}$$

4.6 Best Match 25 (Okapi BM25)

Developed by Stephen Robertson and Karen Spärck Jones at City University London, **BM25** remains the premier state-of-the-art non-linear lexical baseline:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}, \quad (4.8)$$

with Robertson-Spärck Jones IDF:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right). \quad (4.9)$$

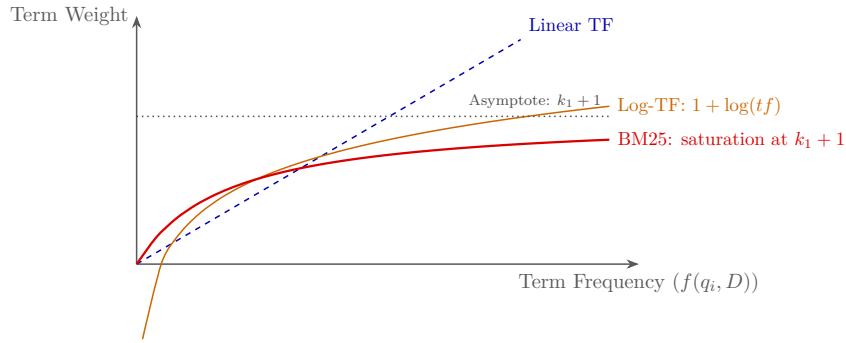


Figure 4.1: Qualitative comparison of linear TF, log-TF, and BM25 asymptotic saturation.

- $k_1 \in [1.2, 2.0]$ governs term frequency saturation, bounded by $\lim_{f \rightarrow \infty} = k_1 + 1$.
- $b \in [0, 1]$ (typically 0.75) governs relative document length normalization against average length avgdl.

Large-Scale BM25 Evaluation

Okapi BM25 defines the scoring function for individual query–document pairs. On web-scale collections containing billions of documents, evaluating BM25 across inverted lists using modern query processing strategies (*Term-at-a-Time* and *Document-at-a-Time* with WAND dynamic pruning) is detailed in Chapter 5.

4.7 Summary Conceptual Map

Figure 4.2 synthesizes the full trajectory from raw text acquisition to vector and probabilistic scoring.

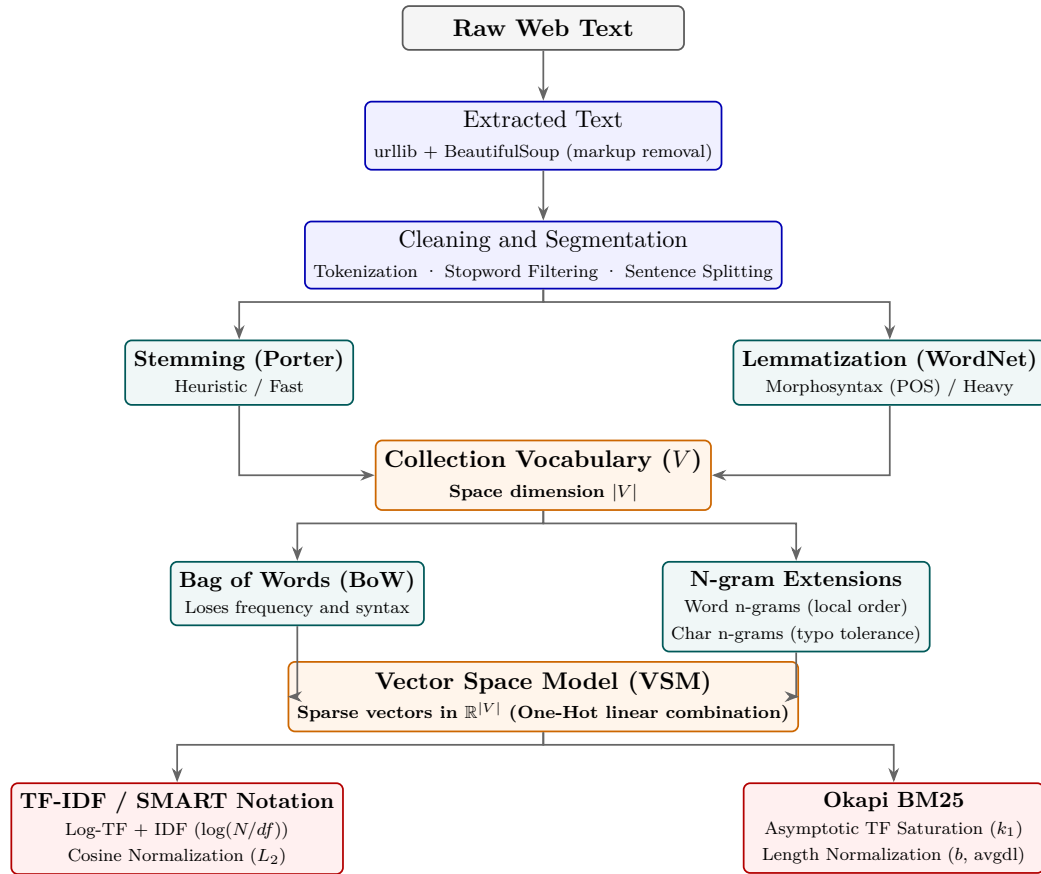


Figure 4.2: Integrated conceptual map of the IR pipeline: from text scraping to vector and probabilistic retrieval models.

Chapter 5

Search Engine Architecture: Indexing and Query Processing

The preceding chapters established the core mathematical foundations of Information Retrieval: evaluation metrics for ranked lists (Chapter 1), linguistic properties of textual data (Chapter 3), and the mathematical derivation of relevance scoring under the Vector Space Model and Okapi BM25 (Chapter 4). These models define *what* must be computed to rank documents with respect to user queries.

This chapter addresses the complementary systems and algorithmic challenge: **how to compute these continuous scores in real time over planetary-scale corpora**, maintaining sub-50 ms latencies across collections containing billions of documents. We examine the two-stage retrieval pipeline, analyze the extreme sparsity of term–document matrices contrasting *Forward Indexes* with *Inverted Indexes*, study memory layout optimizations and integer compression, inspect positional indexes for phrase queries alongside query expansion pathologies, and deliver a detailed algorithmic and architectural comparison between *Term-at-a-Time* (TAAT) and *Document-at-a-Time* (DAAT) query processing.

5.1 System Architecture and the Two-Stage Retrieval Paradigm

Large-scale search engines must satisfy stringent throughput and latency constraints (< 50–100 ms). To achieve this, computation is strictly divided between massive offline batch processes and online interactive query servicing.

5.1.1 The Offline Pipeline: Ingestion, Indexing, and Compression

The offline pipeline processes an unstructured corpus $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ through three stages:

1. **Text Processing and Parsing:** Web crawled documents are stripped of markup, tokenized, and normalized (Section 3.4).
2. **Vocabulary and Index Construction:** The lexicon $\mathcal{V} = \{t_1, t_2, \dots, t_M\}$ is extracted, mapping every unique term to its occurrences across documents.
3. **Lossless Integer Compression:** Because raw index data can rival or exceed the raw text size, integer compression algorithms are applied to ensure index segments reside in high-speed memory tiers (RAM and CPU caches, Chapter 2).

5.1.2 The Two-Stage Retrieval Pipeline

Commercial web search engines resolve queries using a two-stage pipeline:

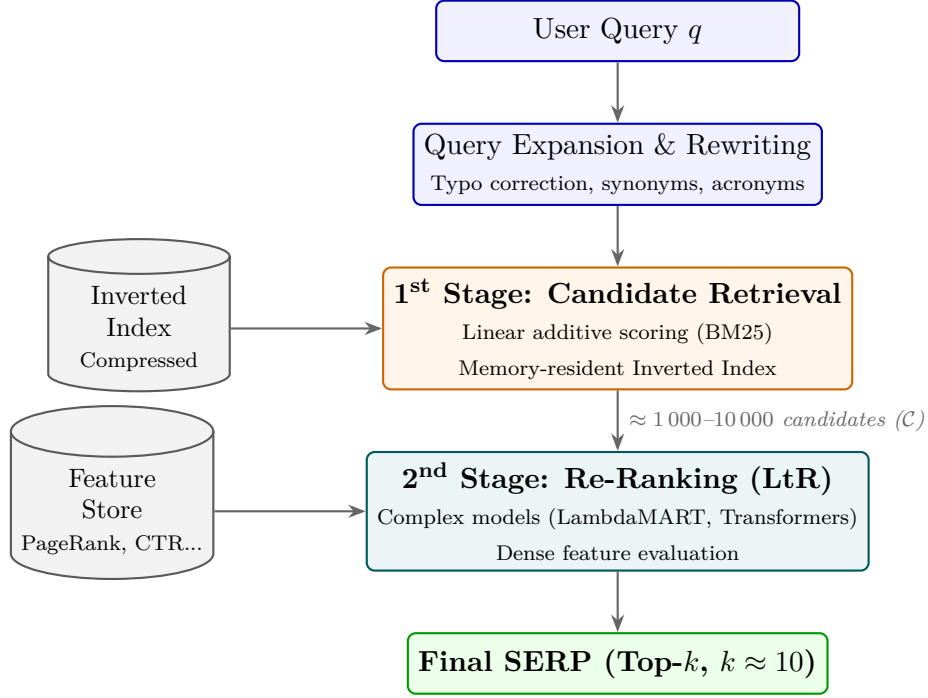


Figure 5.1: Architecture of the two-stage retrieval pipeline: high-throughput coarse candidate generation followed by high-capacity machine-learned re-ranking.

1. **First Stage (Candidate Retrieval):** Operates over the entire corpus ($N \sim 10^9\text{--}10^{10}$ documents) using fast additive scoring functions (Okapi BM25) to isolate a small candidate set:

$$\mathcal{C} \subset \mathcal{D}, \quad |\mathcal{C}| \approx 1\,000\text{--}10\,000 \text{ documents.} \quad (5.1)$$

2. **Second Stage (Re-Ranking / Heavy Scoring):** Operates exclusively on $\mathcal{C}$ using complex Machine Learning models (Learning-to-Rank algorithms such as LambdaMART [4] or cross-encoder Transformers) evaluating hundreds to thousands of dense signals to produce the final top- k ($k \approx 10$).

5.2 Scoring Models: The Dot Product Bridge to Dense Retrieval

5.2.1 Recap: Limitations of Exact Boolean and Jaccard Scoring

As established in Chapter 4 (Sections 4.1 and 4.2), exact Boolean matching suffers from the *Feast or Famine* pathology, while unweighted set-based Jaccard scoring unfairly penalizes longer documents and ignores term frequency (*tf*) and collection specificity (*idf*).

5.2.2 The Dot Product as a Unified Scoring Operator

The Vector Space Model and statistical weighting schemes resolve these limitations by formulating relevance as a vector dot product:

$$\text{Score}(q, d) = \mathbf{q} \cdot \mathbf{d} = \sum_{t \in \mathcal{V}} w(t, q) \cdot w(t, d) = \sum_{t \in q \cap d} w(t, q) \cdot w(t, d). \quad (5.2)$$

This formulation bridges classical lexical retrieval and modern neural dense retrieval:

- **Lexical Sparse Retrieval:** Vectors reside in $\mathbb{R}^{|\mathcal{V}|}$, with dimensions corresponding to exact vocabulary terms ($|\mathcal{V}| \sim 10^7$). Vectors are ultra-sparse, scored using statistical weights (TF-IDF, BM25).
- **Dense Neural Retrieval:** Transformer-based encoders project queries and passages into low-dimensional dense embeddings $\mathbf{v}_q, \mathbf{v}_d \in \mathbb{R}^k$ ($k \approx 768\text{--}1536$). Semantic similarity is determined by vector dot product:

$$\text{Score}_{\text{neural}}(q, d) = \mathbf{v}_q \cdot \mathbf{v}_d = \sum_{i=1}^k v_{q,i} \cdot v_{d,i}. \quad (5.3)$$

In both domains, real-time efficiency depends on aggregating dot products over massive corpora.

5.3 High-Performance Data Structures: Forward Index vs. Inverted Index

5.3.1 Matrix Sparsity and the Dense Bottleneck

For a web-scale search engine:

- Vocabulary size: $|\mathcal{V}| \sim 10^7$ unique terms.
- Document collection size: $|\mathcal{D}| \sim 10^9\text{--}10^{10}$ documents.
- Total matrix cells: $|\mathcal{V}| \times |\mathcal{D}| \approx 10^{16}\text{--}10^{17}$ cells.

Because an average document contains only $10^2\text{--}10^3$ distinct words, the non-zero density is below 0.001%; over 99.99% of matrix entries are identically zero. Allocating a dense matrix would require petabytes of redundant memory and demand an unacceptable $\mathcal{O}(|\mathcal{V}| \cdot |\mathcal{D}|)$ scan per query.

5.3.2 Sparse Serialization: Forward Index vs. Inverted Index

Sparse representations record only non-zero (ID, Value) pairs:

- **Forward Index (Column-Wise / Document-Oriented):** Maps each document to its constituent terms: $\text{DocID} \rightarrow [(t_a, w_a), \dots]$. While memory-efficient, evaluating a query still requires inspecting all N documents ($\mathcal{O}(|\mathcal{D}|)$).
- **Inverted Index (Row-Wise / Term-Oriented):** Maps each term to the ordered sequence of documents containing it: $t \rightarrow [(\text{DocID}_i, w_i), \dots]$. This enables instantaneous static pruning: documents containing none of the query terms are skipped entirely without executing CPU cycles.

Property	Forward Index	Inverted Index
Orientation	Column-wise (Document-Oriented).	Row-wise (Term-Oriented).
Mapping	$\text{DocID} \rightarrow [(t_a, w_a), \dots]$	$t \rightarrow [(\text{DocID}_i, w_i), \dots]$
Query Complexity	$\mathcal{O}(\mathcal{D})$ (full collection scan).	$\mathcal{O}\left(\sum_{t \in q} \mathcal{L}(t) \right)$ (candidate postings only).
Engine Function	Document metadata storage, snippet generation.	Primary indexing core for candidate retrieval.

Table 5.1: Systematic comparison between Forward Index and Inverted Index.

5.4 Inverted Index Anatomy, Memory Layout, and Compression

5.4.1 The Lexicon (Dictionary)

The lexicon stores the vocabulary $\mathcal{V}$, recording for each term t its text string, document frequency df_t (for precomputed IDF), and memory pointer or disk offset to its posting list. The lexicon is kept memory-resident in **RAM** using fast structures (B-Trees, Trie, or front-coded arrays).

5.4.2 Posting Lists: Physical Stream Decoupling

Conceptually, a posting list is a sequence of tuples: $\mathcal{L}(t) = [\langle d_1, tf_1 \rangle, \langle d_2, tf_2 \rangle, \dots]$. In high-performance implementations, storing interleaved structs wastes compression potential. Engines decouple posting lists into two distinct physical streams:

1. **DocID Stream (Strictly Monotonic):** $\mathbf{D}_t = [\text{DocID}_1, \dots, \text{DocID}_m]$ with $\text{DocID}_i < \text{DocID}_{i+1}$. Storing successive differences (d -gaps):

$$\Delta_1 = \text{DocID}_1, \quad \Delta_i = \text{DocID}_i - \text{DocID}_{i-1} \quad (i \geq 2) \quad (5.4)$$

yields small integers compressed via Variable Byte, Elias-Fano, PForDelta, or Binary Interpolative Coding [3].

2. **Term Frequency Stream (Unordered):** $\mathbf{F}_t = [tf_1, \dots, tf_m]$. Values are small positive integers (1, 2, 3) compressed using symmetric variable-length integer codes.

5.4.3 Compressed Integers vs. Precomputed Floats

Search engines universally store **compressed integers** rather than precomputed floating-point scores. Large-scale search systems are **memory-bound** (Chapter 2): RAM-to-CPU transfer latencies (≈ 80 ns) dominate execution time. Compressing postings drastically reduces memory bandwidth demands; the minor CPU arithmetic required to compute BM25 scores on the fly is negligible compared to memory wait cycles.

5.5 Query Types, Positional Indexes, and Query Expansion

5.5.1 Phrase Queries and Positional Indexes

Phrase queries (e.g., "neural network architecture") require terms to appear adjacent ($\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1$). Frequency-only inverted indexes cannot verify this constraint. The engine extends the posting structure into a **Positional Index**:

$$t \longrightarrow \left[\langle \text{DocID}_k, \text{tf}_k, [\text{pos}_1, \dots, \text{pos}_{\text{tf}}] \rangle \right]. \quad (5.5)$$

Positional tracking incurs a $2 \times -3 \times$ storage overhead over non-positional indexes.

5.5.2 Query Expansion and Semantic Drift

Engines expand raw user queries with synonyms, spell corrections, and acronyms. Unconstrained lexical expansion can cause severe *semantic drift*. A classic example in Italian occurs when expanding "babbo francesco":

1. *Babbo* is expanded to its regional synonym *papà* (father).
2. Normalization strips accents, mapping *papà* to *papa* (Pope).
3. The query becomes ("babbo" OR "pap`a" OR "papa") AND "francesco".
4. High PageRank pages for Pope Francis (*Papa Francesco*) overwhelm relevance, making Wikipedia's biography of the Pontiff the top result.

5.5.3 Stemming vs. Query Expansion

While stemming and lemmatization reduce words along morphological lines (e.g., *connecting* $\rightarrow$ *connect*, Section 3.6), they cannot link etymologically distinct synonyms (such as *babbo* and *papà*). Modern engines avoid aggressive indexing-time stemming and instead resolve synonyms dynamically via contextual neural query expansion.

5.6 Query Processing: Term-at-a-Time (TAAT) vs. Document-at-a-Time (DAAT)

5.6.1 Term-at-a-Time (TAAT)

TAAT iterates through posting lists **sequentially, one term at a time**:

1. Allocates a global accumulator array $A[\text{DocID}] \in \mathbb{R}$.
2. Reads the first posting list $\mathcal{L}(t_1)$, updating accumulator scores.
3. Proceeds to $\mathcal{L}(t_2)$, accumulating scores until all query terms are exhausted.
4. Extracts the top- k entries using a min-heap over A .

Algorithm 2: Term-at-a-Time (TAAT)

Input: Query $q = \{t_1, \dots, t_m\}$, Inverted Index, integer k

Output: Min-Heap H of Top- k scored documents

```

1  $A \leftarrow \text{InitializeAccumulatorTable}()$ 
2 foreach term  $t_i \in q$  do
3    $\mathcal{L}_i \leftarrow \text{FetchPostingList}(t_i)$ 
4    $\text{idf}_i \leftarrow \text{ComputeIDF}(t_i)$ 
5   foreach  $\langle d, \text{tf} \rangle \in \mathcal{L}_i$  do
6      $s_{\text{partial}} \leftarrow \text{EvaluateTermScore}(\text{tf}, \text{idf}_i)$ 
7     if  $d \in A$  then
8        $A[d] \leftarrow A[d] + s_{\text{partial}}$ 
9     else
10       $A[d] \leftarrow s_{\text{partial}}$ 
11  $H \leftarrow \text{InitializeMinHeap}(\text{capacity} = k)$ 
12 foreach entry  $(d, \text{score}) \in A$  do
13   if  $H.\text{Size}() < k$  then
14      $H.\text{Insert}(d, \text{score})$ 
15   else
16     if  $\text{score} > H.\text{MinScore}()$  then
17        $H.\text{ExtractMin}()$ 
18        $H.\text{Insert}(d, \text{score})$ 
19 return  $H$ 

```

Characteristics: Excellent sequential memory locality and hardware prefetching, but suffers from explosive RAM accumulator footprints and cannot perform dynamic early pruning.

5.6.2 Document-at-a-Time (DAAT)

DAAT processes all query posting lists ****in parallel across synchronized pointers****:

1. Initializes cursors p_i at the head of each list $\mathcal{L}(t_i)$.
2. Identifies the current minimum document identifier: $d_{\min} = \min_i \{p_i.\text{CurrentDocID}()\}$.
3. Accumulates the full final score $S(q, d_{\min})$ across all cursors matching $d_{\min}$.
4. Updates the fixed-size top- k min-heap immediately.
5. Advances all cursors matching $d_{\min}$ to their next entry.

Algorithm 3: Document-at-a-Time (DAAT)

Input: Query $q = \{t_1, \dots, t_m\}$, Inverted Index, integer k

Output: Min-Heap H of Top- k scored documents

```

1 cursors  $\leftarrow$  InitializeListCursors( $q$ )
2  $H \leftarrow$  InitializeMinHeap(capacity =  $k$ )
3 while HasActiveCursors(cursors) do
4    $d_{\min} \leftarrow \min\{p.\text{CurrentDocID}() \mid p \in \text{cursors} \wedge p.\text{IsActive}()\}$ 
5    $\text{score}_{\text{tot}} \leftarrow 0.0$ 
6   foreach  $p \in \text{cursors}$  do
7     if  $p.\text{IsActive}() \wedge p.\text{CurrentDocID}() == d_{\min}$  then
8        $\text{score}_{\text{tot}} \leftarrow$ 
9          $\text{score}_{\text{tot}} + \text{EvaluateTermScore}(p.\text{CurrentTF}(), p.\text{Term}())$ 
10       $p.\text{Advance}()$ 
11   if  $H.\text{Size}() < k$  then
12      $H.\text{Insert}(d_{\min}, \text{score}_{\text{tot}})$ 
13   else
14     if  $\text{score}_{\text{tot}} > H.\text{MinScore}()$  then
15        $H.\text{ExtractMin}()$ 
16        $H.\text{Insert}(d_{\min}, \text{score}_{\text{tot}})$ 
17 return  $H$ 

```

Characteristics: Minimal working memory ($\mathcal{O}(k)$ heap and $\mathcal{O}(m)$ cursor state) and native support for dynamic early termination algorithms (WAND and Block-Max WAND), at the cost of non-contiguous interleaved memory access patterns.

Dimension	Term-at-a-Time (TAAT)	Document-at-a-Time (DAAT)
Traversal Direction	Horizontal (term-by-term, sequential).	Vertical (document-by-document, synchronous).
Score State	Partial progressive accumulators.	Fully consolidated score per document.
RAM Footprint	High ($\mathcal{O}(\bigcup \mathcal{L}(t_i))$ accumulators).	Minimal ($\mathcal{O}(k)$ for Top- k min-heap).
Cache Locality	Excellent (contiguous stream scans).	Interleaved (pointer jumping across lists).
Early Termination	Ineffective.	Native and powerful (WAND, Block-Max).
Industry Role	GPU and SIMD accelerator architectures.	Core CPU search engines (Lucene, Elasticsearch).

Table 5.2: Systematic comparison between TAAT and DAAT query processing strategies.

5.6.3 Significance in Modern Vector and Hybrid Search

The TAAT vs. DAAT paradigm extends directly into modern retrieval systems:

- **Sparse Neural Retrieval (e.g., SPLADE [6]):** Employs inverted indexes over expanded neural vocabularies evaluated using DAAT with Block-Max WAND pruning.
- **Hardware Acceleration:** GPU-based vector accelerators leverage TAAT-style batching to exploit streaming multiprocessors and high memory bandwidth.
- **Dedicated Vector Engines (e.g., Qdrant):** Modern engines combine approximate nearest neighbor graphs (k -ANN on HNSW) with inverted indexes for payload filtering, combining DAAT and TAAT principles for hybrid multi-stage retrieval.

Bibliography

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.